

ASSIGNMENT – 1 & 2

HELLO WORLD - CONSOLE OUTPUT

```
// Program 1: Basic console output
console.log("Hello, JavaScript World!");
console.log("This is my first JS program.");
```

COMMENTS AND BASIC SYNTAX

```
// Program 2: Understanding comments and statements
```

```
// This is a single-line comment
```

```
/*
```

```
    This is a multi-line comment
```

```
    It can span multiple lines
```

```
*/
```

```
let message = "Learning JavaScript"; // End of statement with semicolon
console.log(message);                // Output: Learning JavaScript
```

VARIABLES WITH VAR, LET, AND CONST

```
// Program 3: Variable declarations and scope differences
```

```
// var - function scoped (older way)
```

```
var oldWay = "I am declared with var";
```

```
console.log("var variable:", oldWay);
```

```
// let - block scoped (modern way)
```

```
let modernWay = "I am declared with let";
```

```
console.log("let variable:", modernWay);
```

```
// const - cannot be reassigned
```

ASSIGNMENT – 1 & 2

```
const constantValue = "I cannot change";  
console.log("const variable:", constantValue);  
// Uncommenting below line would cause error:  
// constantValue = "New value"; // TypeError: Assignment to constant  
variable
```

DATA TYPES AND TYPEOF OPERATOR

// Program 4: Exploring JavaScript data types

```
let numberType = 42;  
let stringType = "Hello World";  
let booleanType = true;  
let undefinedType;  
let nullType = null;  
let objectType = { name: "John" };  
let arrayType = [1, 2, 3];  
let functionType = function() { return "Hello"; };  
  
console.log("number:", typeof numberType);    // "number"  
console.log("string:", typeof stringType);    // "string"  
console.log("boolean:", typeof booleanType);  // "boolean"  
console.log("undefined:", typeof undefinedType); // "undefined"  
console.log("null:", typeof nullType);        // "object" (JavaScript quirk!)  
console.log("object:", typeof objectType);    // "object"  
console.log("array:", typeof arrayType);      // "object" (arrays are objects)
```

ASSIGNMENT – 1 & 2

```
console.log("function:", typeof functionType); // "function"
```

STRING OPERATIONS AND CONCATENATION

```
// Program 5: Working with strings
```

```
let firstName = "John";
```

```
let lastName = "Doe";
```

```
let age = 25;
```

```
// Method 1: Concatenation with +
```

```
let fullName1 = firstName + " " + lastName;
```

```
console.log("Concatenation:", fullName1);
```

```
// Method 2: Template Literals (ES6+) - Preferred
```

```
let fullName2 = `${firstName} ${lastName} is ${age} years old`;
```

```
console.log("Template Literal:", fullName2);
```

```
// String methods
```

```
let text = "JavaScript Programming";
```

```
console.log("Length:", text.length); // 22
```

```
console.log("Uppercase:", text.toUpperCase()); // JAVASCRIPT  
PROGRAMMING
```

```
console.log("Lowercase:", text.toLowerCase()); // javascript  
programming
```

```
console.log("Substring:", text.substring(0, 10)); // JavaScript
```

```
console.log("Includes 'Script':", text.includes("Script")); // true
```

ASSIGNMENT – 1 & 2

```
console.log("Character at index 5:", text.charAt(5)); // c
```

ARITHMETIC OPERATIONS

```
// Program 6: Basic arithmetic operations
```

```
let a = 15;
```

```
let b = 4;
```

```
console.log("Addition:", a + b); // 19
```

```
console.log("Subtraction:", a - b); // 11
```

```
console.log("Multiplication:", a * b); // 60
```

```
console.log("Division:", a / b); // 3.75
```

```
console.log("Modulus (Remainder):", a % b); // 3
```

```
console.log("Exponentiation:", a ** b); // 50625 (15 ^ 4)
```

```
// Increment and Decrement
```

```
let counter = 10;
```

```
console.log("Original:", counter); // 10
```

```
console.log("Post-increment:", counter++); // 10 (returns then increments)
```

```
console.log("After post-increment:", counter); // 11
```

```
console.log("Pre-increment:", ++counter); // 12 (increments then returns)
```

```
console.log("Post-decrement:", counter--); // 12
```

```
console.log("After post-decrement:", counter); // 11
```

COMPARISON OPERATORS

```
// Program 7: Understanding comparison operators
```

```
let x = 5;
```

```
let y = "5";
```

ASSIGNMENT – 1 & 2

```
let z = 10;
```

```
console.log("Equal (==):", x == y);      // true (loose equality)
console.log("Strict Equal (===):", x === y); // false (strict equality)
console.log("Not Equal (!=):", x != y);   // false
console.log("Strict Not Equal (!==):", x !== y); // true
```

```
console.log("Greater than:", z > x);      // true
console.log("Less than:", x < z);         // true
console.log("Greater or equal:", x >= 5); // true
console.log("Less or equal:", z <= 10);   // true
```

LOGICAL OPERATORS

```
// Program 8: AND, OR, NOT operators
```

```
let isLoggedIn = true;
let isAdmin = false;
let hasPermission = true;
```

```
// AND (&&) - Both must be true
```

```
console.log("Admin access:", isLoggedIn && isAdmin); // false
```

```
// OR (||) - At least one must be true
```

```
console.log("Can view page:", isLoggedIn || hasPermission); // true
```

ASSIGNMENT – 1 & 2

// NOT (!) - Inverts boolean

```
console.log("Is NOT admin:", !isAdmin); // true
```

// Short-circuit evaluation

```
let userName = "";
```

```
let displayName = userName || "Guest User";
```

```
console.log("Display name:", displayName); // "Guest User"
```

// Nullish coalescing (??) - Only checks null/undefined

```
let count = 0;
```

```
let result = count ?? 10;
```

```
console.log("Nullish result:", result); // 0 (not 10, because 0 is not null/undefined)
```

TYPE COERCION AND CONVERSION

// Program 9: Understanding automatic and manual type conversion

// Implicit Coercion (Automatic)

```
console.log("5" + 3);    // "53" (string concatenation)
```

```
console.log("5" - 3);    // 2 (numeric subtraction)
```

```
console.log("5" * "2");  // 10 (numeric multiplication)
```

```
console.log(true + true); // 2 (true is 1, false is 0)
```

// Explicit Conversion (Manual)

```
let strNumber = "123.45";
```

```
let intNumber = parseInt(strNumber);
```

```
let floatNumber = parseFloat(strNumber);
```

ASSIGNMENT – 1 & 2

```
let numberConverted = Number(strNumber);
console.log("parseInt:", parseInt(strNumber)); // 123
console.log("parseFloat:", parseFloat(strNumber)); // 123.45
console.log("Number():", Number(strNumber)); // 123.45
// Converting to String
let num = 456;
console.log("toString():", num.toString()); // "456"
console.log("String():", String(num)); // "456"
// Converting to Boolean
console.log("Boolean(1):", Boolean(1)); // true
console.log("Boolean(0):", Boolean(0)); // false
console.log("Boolean(''):", Boolean('')); // false
console.log("Boolean('hello'):", Boolean('hello')); // true
```

USER INPUT WITH PROMPT()

// Program 10: Getting input from user via prompt dialog

```
let userName = prompt("Please enter your name:");
let userAge = prompt("Please enter your age:");

// Convert age string to number
let ageNumber = parseInt(userAge);

// Calculate birth year
let currentYear = 2026;
```

ASSIGNMENT – 1 & 2

```
let birthYear = currentYear - ageNumber;
```

```
// Display output using alert
```

```
alert(`Hello ${userName}! You were born around ${birthYear}.`);
```

```
// Also log to console
```

```
console.log(`User: ${userName}, Age: ${ageNumber}, Birth Year: ${birthYear}`);
```

IF-ELSE STATEMENT

```
// Program 11: Conditional execution with if-else
```

```
let score = parseInt(prompt("Enter your test score (0-100):"));
```

```
if (score >= 90) {
```

```
    console.log("Grade: A - Excellent!");
```

```
} else if (score >= 80) {
```

```
    console.log("Grade: B - Very Good!");
```

```
} else if (score >= 70) {
```

```
    console.log("Grade: C - Good!");
```

```
} else if (score >= 60) {
```

```
    console.log("Grade: D - Passing!");
```

```
} else {
```

```
    console.log("Grade: F - Needs Improvement.");
```

```
}
```


ASSIGNMENT – 1 & 2

```
console.log(`Your score: ${score}`);
```

NESTED IF-ELSE

```
// Program 12: Nested conditional statements
```

```
let age = parseInt(prompt("Enter your age:"));  
let hasLicense = prompt("Do you have a driver's license?  
(yes/no)").toLowerCase();
```

```
if (age >= 18) {  
    console.log("You are old enough to drive.");  
  
    if (hasLicense === "yes") {  
        console.log("You can legally drive a car.");  
    } else if (hasLicense === "no") {  
        console.log("You need to get a license first.");  
    } else {  
        console.log("Invalid input for license.");  
    }  
} else if (age >= 16) {  
    console.log("You can get a learner's permit.");  
  
    if (hasLicense === "yes") {  
        console.log("You may drive with supervision.");  
    } else {
```

ASSIGNMENT – 1 & 2

```
        console.log("Apply for a learner's permit.");
    }
} else {
    console.log("You are too young to drive.");
}
```

SWITCH STATEMENT

// Program 13: Multiple conditions with switch

```
let dayNumber = parseInt(prompt("Enter a day number (1-7):"));
let dayName;
switch(dayNumber) {
    case 1:
        dayName = "Monday";
        break;
    case 2:
        dayName = "Tuesday";
        break;
    case 3:
        dayName = "Wednesday";
        break;
    case 4:
        dayName = "Thursday";
        break;
    case 5:
        dayName = "Friday";
        break;
```

ASSIGNMENT – 1 & 2

case 6:

```
    dayName = "Saturday";
```

```
    break;
```

case 7:

```
    dayName = "Sunday";
```

```
    break;
```

default:

```
    dayName = "Invalid day number!";
```

```
}
```

```
console.log(`Day ${dayNumber} is ${dayName}`);
```

// Switch with multiple cases

```
let grade = prompt("Enter letter grade (A, B, C, D, F):").toUpperCase();
```

```
let message;
```

```
switch(grade) {
```

```
    case "A":
```

```
        message = "Outstanding!";
```

```
        break;
```

```
    case "B":
```

```
    case "C":
```

```
        message = "Good job!";
```

```
        break;
```

```
    case "D":
```

```
        message = "You passed.";
```

ASSIGNMENT – 1 & 2

```
    break;
case "F":
    message = "Better luck next time.";
    break;
default:
    message = "Invalid grade.";
}
```

```
console.log(message);
```

FOR LOOP - BASIC ITERATION

```
// Program 14: Introduction to for loops
```

```
console.log("Counting from 1 to 10:");
for (let i = 1; i <= 10; i++) {
    console.log(i);
}

console.log("\nCounting backwards from 10 to 1:");
for (let i = 10; i >= 1; i--) {
    console.log(i);
}

console.log("\nEven numbers from 2 to 20:");
for (let i = 2; i <= 20; i += 2) {
    console.log(i);
}
```

```
// Multiplication table
```

ASSIGNMENT – 1 & 2

```
let number = parseInt(prompt("Enter a number for multiplication table:"));
console.log(`\nMultiplication table for ${number}:`);
for (let i = 1; i <= 10; i++) {
    console.log(`${number} x ${i} = ${number * i}`);
}
```

NESTED FOR LOOPS

// Program 15: Nested loops for patterns

```
console.log("Pattern 1: Right Triangle");
for (let i = 1; i <= 5; i++) {
    let row = "";
    for (let j = 1; j <= i; j++) {
        row += "* ";
    }
    console.log(row);
}

console.log("\nPattern 2: Multiplication Grid (1-5)");
for (let i = 1; i <= 5; i++) {
    let row = "";
    for (let j = 1; j <= 5; j++) {
        row += (i * j).toString().padStart(3, " ");
    }
    console.log(row);
}

console.log("\nPattern 3: Pyramid");
let rows = 5;
```

ASSIGNMENT – 1 & 2

```
for (let i = 1; i <= rows; i++) {  
    let spaces = " ".repeat(rows - i);  
    let stars = "*".repeat(2 * i - 1);  
    console.log(spaces + stars);  
}
```

WHILE LOOP

// Program 16: While loop for unknown iterations

// Example 1: Countdown

```
console.log("Countdown:");  
let countdown = 5;  
while (countdown > 0) {  
    console.log(countdown);  
    countdown--;  
}  
console.log("Blast off!");
```

// Example 2: Sum until user enters 0

```
console.log("\nSum Calculator (Enter 0 to stop:");  
let sum = 0;  
let input = 1;  
  
while (input !== 0) {  
    input = parseInt(prompt("Enter a number (0 to exit):"));  
    sum += input;
```

ASSIGNMENT – 1 & 2

```
    console.log(`Current sum: ${sum}`);
}
console.log(`Final sum: ${sum}`);

// Example 3: Number guessing game
let secretNumber = Math.floor(Math.random() * 10) + 1;
let guess;
let attempts = 0;

while (guess !== secretNumber) {
    guess = parseInt(prompt("Guess the number (1-10):"));
    attempts++;

    if (guess < secretNumber) {
        console.log("Too low! Try again.");
    } else if (guess > secretNumber) {
        console.log("Too high! Try again.");
    }
}

console.log(`Correct! The number was ${secretNumber}.`);
console.log(`It took you ${attempts} attempts.`);
```

ASSIGNMENT – 1 & 2

DO-WHILE LOOP

// Program 17: Do-while loop (executes at least once)

// Example 1: Menu system

let choice;

do {

 console.log("\n=== MENU ===");

 console.log("1. Say Hello");

 console.log("2. Show Date");

 console.log("3. Show Time");

 console.log("4. Exit");

 choice = parseInt(prompt("Enter your choice (1-4):"));

switch(choice) {

 case 1:

 console.log("Hello there!");

 break;

 case 2:

 console.log("Date:", new Date().toLocaleDateString());

 break;

 case 3:

 console.log("Time:", new Date().toLocaleTimeString());

 break;

 case 4:

ASSIGNMENT – 1 & 2

```
    console.log("Goodbye!");  
    break;  
default:  
    console.log("Invalid choice. Try again.");  
}  
} while (choice !== 4);
```

// Example 2: Input validation

```
let password;  
do {  
    password = prompt("Enter password (min 6 characters):");  
    if (password.length < 6) {  
        console.log("Password too short! Must be at least 6 characters.");  
    }  
} while (password.length < 6);  
console.log("Password accepted!");
```

BREAK AND CONTINUE

// Program 18: Controlling loop flow with break and continue

```
console.log("Break Example - Stop at 5:");  
for (let i = 1; i <= 10; i++) {  
    if (i === 5) {  
        console.log("Breaking at i =", i);  
        break; // Exit loop completely
```

ASSIGNMENT – 1 & 2

```
}  
  console.log(i);  
}
```

```
console.log("\nContinue Example - Skip multiples of 3:");  
for (let i = 1; i <= 10; i++) {  
  if (i % 3 === 0) {  
    continue; // Skip to next iteration  
  }  
  console.log(i);  
}
```

```
console.log("\nPrime Number Finder:");  
let n = 20;  
console.log(`Prime numbers up to ${n}:`);
```

```
outerLoop: for (let i = 2; i <= n; i++) {  
  for (let j = 2; j < i; j++) {  
    if (i % j === 0) {  
      continue outerLoop; // Skip to next i (labeled continue)  
    }  
  }  
  console.log(i);  
}
```

[ERROR HANDLING WITH TRY-CATCH](#)

ASSIGNMENT – 1 & 2

// Program 19: Handling errors gracefully

// Example 1: Basic try-catch

```
try {  
    console.log("Trying to execute code...");  
    let result = 10 / 0;  
    console.log("Result:", result);  
  
    // This will cause an error  
    let obj = undefined;  
    console.log(obj.property); // TypeError  
} catch (error) {  
    console.log("An error occurred:", error.message);  
    console.log("Error name:", error.name);  
} finally {  
    console.log("This always executes.");  
}
```

// Example 2: Input validation with try-catch

```
function divideNumbers(a, b) {  
    try {  
        if (b === 0) {  
            throw new Error("Division by zero is not allowed!");  
        }  
        if (typeof a !== "number" || typeof b !== "number") {  
            throw new TypeError("Both arguments must be numbers!");  
        }  
    }  
}
```

ASSIGNMENT – 1 & 2

```
        return a / b;
    } catch (error) {
        console.log("Error in divideNumbers:", error.message);
        return null;
    }
}

console.log("\nDivision Tests:");
console.log("10 / 2 =", divideNumbers(10, 2));    // 5
console.log("10 / 0 =", divideNumbers(10, 0));    // null (error handled)
console.log("'a' / 2 =", divideNumbers("a", 2));  // null (error handled)
```

CUSTOM ERROR TYPES

// Program 20: Creating and using custom errors

```
class ValidationError extends Error {
    constructor(message, field) {
        super(message);
        this.name = "ValidationError";
        this.field = field;
    }
}

class AgeError extends ValidationError {
    constructor(message, age) {
        super(message, "age");
    }
}
```

ASSIGNMENT – 1 & 2

```
    this.age = age;
  }
}
```

```
function validateUser(name, age) {
  try {
    if (!name || name.trim() === "") {
      throw new ValidationError("Name cannot be empty", "name");
    }

    if (typeof age !== "number" || isNaN(age)) {
      throw new ValidationError("Age must be a number", "age");
    }

    if (age < 0) {
      throw new AgeError("Age cannot be negative", age);
    }

    if (age < 18) {
      throw new AgeError("User must be 18 or older", age);
    }

    console.log(`User ${name} (${age}) is valid.`);
    return true;
  }
}
```

ASSIGNMENT – 1 & 2

```
} catch (error) {  
  if (error instanceof AgeError) {  
    console.log(`Age Error (${error.age}): ${error.message}`);  
  } else if (error instanceof ValidationError) {  
    console.log(`Validation Error (${error.field}): ${error.message}`);  
  } else {  
    console.log("Unknown Error:", error.message);  
  }  
  return false;  
}  
}
```

```
// Test the validation  
console.log("\nTesting User Validation:");  
validateUser("John Doe", 25); // Valid  
validateUser("", 25); // Name error  
validateUser("Jane", -5); // Age error  
validateUser("Bob", 15); // Age error  
validateUser("Alice", "twenty"); // Type error
```

FUNCTION DECLARATIONS

```
// Program 21: Basic function declarations and invocation
```

```
// Function without parameters
```

```
function greet() {  
  console.log("Hello, welcome to JavaScript!");  
}
```

ASSIGNMENT – 1 & 2

```
}
```

```
// Function with parameters
```

```
function greetPerson(name) {  
    console.log(`Hello, ${name}! Welcome to JavaScript!`);  
}
```

```
// Function with return value
```

```
function add(a, b) {  
    return a + b;  
}
```

```
// Function with multiple returns
```

```
function getGrade(score) {  
    if (score >= 90) return "A";  
    if (score >= 80) return "B";  
    if (score >= 70) return "C";  
    if (score >= 60) return "D";  
    return "F";  
}
```

```
// Calling functions
```

```
greet();  
greetPerson("Alice");  
greetPerson("Bob");
```

ASSIGNMENT – 1 & 2

```
let sum = add(15, 25);  
console.log("15 + 25 =", sum);
```

```
console.log("Score 95 gets grade:", getGrade(95));  
console.log("Score 75 gets grade:", getGrade(75));  
console.log("Score 55 gets grade:", getGrade(55));
```

FUNCTION EXPRESSIONS AND HOISTING

// Program 22: Different ways to define functions

// Function Declaration - Hoisted (can be called before definition)

```
console.log("Hoisting Example:");  
console.log("multiply(4, 5) =", multiply(4, 5)); // Works!
```

```
function multiply(a, b) {  
    return a * b;  
}
```

// Function Expression - NOT hoisted

// console.log(subtract(10, 4)); // ERROR: Cannot access before initialization

```
const subtract = function(a, b) {  
    return a - b;  
};
```


ASSIGNMENT – 1 & 2

```
console.log("subtract(10, 4) =", subtract(10, 4)); // Works here
```

```
// Named Function Expression
```

```
const factorial = function fact(n) {  
  if (n <= 1) return 1;  
  return n * fact(n - 1); // 'fact' is only visible inside the function  
};
```

```
console.log("factorial(5) =", factorial(5));
```

```
// console.log(fact(5)); // ERROR: fact is not defined
```

ARROW FUNCTIONS

```
// Program 23: ES6 Arrow functions
```

```
// Traditional function
```

```
const traditionalAdd = function(a, b) {  
  return a + b;  
};
```

```
// Arrow function - basic syntax
```

```
const arrowAdd = (a, b) => {  
  return a + b;  
};
```

```
// Arrow function - implicit return (no curly braces)
```

ASSIGNMENT – 1 & 2

```
const implicitAdd = (a, b) => a + b;
```

```
// Arrow function - single parameter (no parentheses needed)
```

```
const square = x => x * x;
```

```
// Arrow function - no parameters
```

```
const getRandom = () => Math.floor(Math.random() * 100) + 1;
```

```
// Arrow function - returning an object (wrap in parentheses)
```

```
const createPerson = (name, age) => ({ name: name, age: age });
```

```
console.log("Traditional:", traditionalAdd(5, 3));
```

```
console.log("Arrow with braces:", arrowAdd(5, 3));
```

```
console.log("Arrow implicit:", implicitAdd(5, 3));
```

```
console.log("Square of 7:", square(7));
```

```
console.log("Random number:", getRandom());
```

```
console.log("Created person:", createPerson("John", 30));
```

PARAMETERS AND DEFAULT VALUES

```
// Program 24: Function parameters and default values
```

```
// Default parameters (ES6+)
```

```
function greet(name = "Guest", greeting = "Hello") {
```

```
    console.log(` ${greeting}, ${name}!`);
```

```
}
```

ASSIGNMENT – 1 & 2

```
greet();           // Hello, Guest!
greet("Alice");    // Hello, Alice!
greet("Bob", "Hi"); // Hi, Bob!
greet(undefined, "Welcome"); // Welcome, Guest!
```

```
// Rest parameters (...)
```

```
function sum(...numbers) {
  let total = 0;
  for (let num of numbers) {
    total += num;
  }
  return total;
}
```

```
console.log("Sum of 1,2,3:", sum(1, 2, 3));
console.log("Sum of 1,2,3,4,5:", sum(1, 2, 3, 4, 5));
```

```
// Arguments object (older way)
```

```
function oldStyleSum() {
  let total = 0;
  for (let i = 0; i < arguments.length; i++) {
    total += arguments[i];
  }
  return total;
}
```

ASSIGNMENT – 1 & 2

```
console.log("Old style sum:", oldStyleSum(1, 2, 3, 4));
```

SCOPE AND CLOSURES

```
// Program 25: Understanding scope and closures
```

```
// Global scope
```

```
let globalVar = "I am global";
```

```
function outerFunction(outerParam) {
```

```
    // Function scope (local to outerFunction)
```

```
    let outerVar = "I am in outer function";
```

```
    function innerFunction(innerParam) {
```

```
        // Function scope (local to innerFunction)
```

```
        let innerVar = "I am in inner function";
```

```
        console.log("Inner can access:");
```

```
        console.log("- globalVar:", globalVar);
```

```
        console.log("- outerParam:", outerParam);
```

```
        console.log("- outerVar:", outerVar);
```

```
        console.log("- innerParam:", innerParam);
```

```
        console.log("- innerVar:", innerVar);
```

```
    }
```

```
    innerFunction("inner argument");
```

ASSIGNMENT – 1 & 2

```
// console.log(innerVar); // ERROR: Not accessible here
}
```

```
outerFunction("outer argument");
```

// Closure Example - Counter Factory

```
function createCounter(initialValue = 0) {
  let count = initialValue; // Private variable

  return {
    increment: function() {
      count++;
      return count;
    },
    decrement: function() {
      count--;
      return count;
    },
    getValue: function() {
      return count;
    },
    reset: function() {
      count = initialValue;
      return count;
    }
  }
}
```

ASSIGNMENT – 1 & 2

```
};  
}
```

```
const counter1 = createCounter(10);  
const counter2 = createCounter(0);
```

```
console.log("\nCounter 1:");  
console.log(counter1.increment()); // 11  
console.log(counter1.increment()); // 12  
console.log(counter1.getValue()); // 12
```

```
console.log("\nCounter 2:");  
console.log(counter2.decrement()); // -1  
console.log(counter2.getValue()); // -1
```

// The 'count' variable is private and persists between calls!

[CALLBACK FUNCTIONS](#)

// Program 26: Functions as arguments (callbacks)

```
// Function that accepts a callback  
function processArray(arr, callback) {  
    let result = [];  
    for (let i = 0; i < arr.length; i++) {  
        result.push(callback(arr[i], i));  
    }  
}
```

ASSIGNMENT – 1 & 2

```
    return result;
}
```

```
// Different callbacks
```

```
function double(x) {
    return x * 2;
}
```

```
function square(x) {
    return x * x;
}
```

```
function formatCurrency(x) {
    return `$$${x.toFixed(2)}`;
}
```

```
let numbers = [1, 2, 3, 4, 5];
console.log("Original array:", numbers);
console.log("Doubled:", processArray(numbers, double));
console.log("Squared:", processArray(numbers, square));
```

```
let prices = [10.5, 25.75, 99.99];
console.log("Prices:", processArray(prices, formatCurrency));
```

```
// Inline callback (anonymous function)
```

ASSIGNMENT – 1 & 2

```
let tripled = processArray(numbers, function(x) {  
    return x * 3;  
});  
console.log("Tripled:", tripled);
```

// Arrow function callback

```
let halved = processArray(numbers, x => x / 2);  
console.log("Halved:", halved);
```

HIGHER-ORDER FUNCTIONS

// Program 27: Functions that return functions

// Function factory

```
function multiplyBy(factor) {  
    return function(number) {  
        return number * factor;  
    };  
}
```

```
const double = multiplyBy(2);
```

```
const triple = multiplyBy(3);
```

```
const quadruple = multiplyBy(4);
```

```
console.log("Double 5:", double(5)); // 10
```

```
console.log("Triple 5:", triple(5)); // 15
```

```
console.log("Quadruple 5:", quadruple(5)); // 20
```


ASSIGNMENT – 1 & 2

// Function composition

```
function compose(f, g) {  
  return function(x) {  
    return f(g(x));  
  };  
}
```

```
const addOne = x => x + 1;
```

```
const multiplyByTwo = x => x * 2;
```

```
const addOneThenDouble = compose(multiplyByTwo, addOne);
```

```
const doubleThenAddOne = compose(addOne, multiplyByTwo);
```

```
console.log("\nFunction Composition:");
```

```
console.log("addOneThenDouble(5):", addOneThenDouble(5)); // (5+1)*2  
= 12
```

```
console.log("doubleThenAddOne(5):", doubleThenAddOne(5)); // (5*2)+1  
= 11
```

// Currying

```
function curryAdd(a) {  
  return function(b) {  
    return function(c) {  
      return a + b + c;  
    }  
  }  
}
```

ASSIGNMENT – 1 & 2

```
};  
};  
}
```

```
console.log("\nCurrying:");  
console.log("curryAdd(1)(2)(3) =", curryAdd(1)(2)(3)); // 6
```

```
const addFive = curryAdd(5);  
const addFiveAndTen = addFive(10);  
console.log("addFiveAndTen(15) =", addFiveAndTen(15)); // 30
```

RECURSIVE FUNCTIONS

// Program 28: Functions that call themselves

// Factorial using recursion

```
function factorial(n) {  
    // Base case  
    if (n <= 1) return 1;  
    // Recursive case  
    return n * factorial(n - 1);  
}
```

```
console.log("Factorial Examples:");  
console.log("5! =", factorial(5)); // 120  
console.log("7! =", factorial(7)); // 5040
```

ASSIGNMENT – 1 & 2

```
// Fibonacci sequence
```

```
function fibonacci(n) {  
    if (n <= 1) return n;  
    return fibonacci(n - 1) + fibonacci(n - 2);  
}
```

```
console.log("\nFibonacci Sequence:");  
for (let i = 0; i <= 10; i++) {  
    console.log(`F(${i}) = ${fibonacci(i)}`);  
}
```

```
// Recursive directory traversal simulation
```

```
function traverseObject(obj, path = "") {  
    for (let key in obj) {  
        let currentPath = path ? `${path}.${key}` : key;  
        if (typeof obj[key] === "object" && obj[key] !== null) {  
            traverseObject(obj[key], currentPath);  
        } else {  
            console.log(`${currentPath} = ${obj[key]}`);  
        }  
    }  
}
```

```
let data = {
```

ASSIGNMENT – 1 & 2

```
user: {  
  name: "John",  
  address: {  
    street: "123 Main St",  
    city: "Boston"  
  }  
},  
settings: {  
  theme: "dark"  
}  
};  
  
console.log("\nRecursive Object Traversal:");  
traverseObject(data);
```

IMMEDIATELY INVOKED FUNCTION EXPRESSIONS (IIFE)

// Program 29: IIFE for creating private scope

// Basic IIFE

```
(function() {  
  let privateVar = "I am private";  
  console.log("IIFE executed:", privateVar);  
})(); // Immediately invoked
```

// IIFE with parameters

```
(function(name, age) {  
  console.log(`Hello ${name}, you are ${age} years old.`);  
})
```

ASSIGNMENT – 1 & 2

```
})( "Alice", 30);
```

```
// IIFE with arrow function
```

```
() => {  
    console.log("Arrow IIFE executed");  
})();
```

```
// Module pattern using IIFE
```

```
const calculator = (function() {  
    // Private variables and functions  
    let result = 0;  
  
    function validate(n) {  
        return typeof n === "number" && !isNaN(n);  
    }  

```

```
// Public API
```

```
return {  
    add: function(n) {  
        if (validate(n)) result += n;  
        return this;  
    },  
    subtract: function(n) {  
        if (validate(n)) result -= n;  
        return this;  
    }  
};
```

ASSIGNMENT – 1 & 2

```
    },  
    multiply: function(n) {  
        if (validate(n)) result *= n;  
        return this;  
    },  
    getResult: function() {  
        return result;  
    },  
    reset: function() {  
        result = 0;  
        return this;  
    }  
};  
})();  
  
console.log("\nCalculator Module:");  
calculator.add(10).subtract(3).multiply(2);  
console.log("Result:", calculator.getResult()); // 14  
calculator.reset().add(5).multiply(4);  
console.log("Result:", calculator.getResult()); // 20  
  
// Result variable is private - cannot be accessed directly  
// console.log(calculator.result); // undefined
```

ASSIGNMENT – 1 & 2

FUNCTION METHODS - CALL, APPLY, BIND

// Program 30: Controlling 'this' with call, apply, and bind

```
const person1 = {  
  firstName: "John",  
  lastName: "Doe",  
  getFullName: function(greeting, punctuation) {  
    return `${greeting} ${this.firstName} ${this.lastName}${punctuation}`;  
  }  
};
```

```
const person2 = {  
  firstName: "Jane",  
  lastName: "Smith"  
};
```

```
// call() - Invokes function with given 'this' and arguments individually  
console.log("Using call():");  
console.log(person1.getFullName.call(person2, "Hello", "!"));  
// Hello Jane Smith!
```

```
// apply() - Same as call, but arguments passed as array  
console.log("\nUsing apply():");  
console.log(person1.getFullName.apply(person2, ["Hi", "!"]));  
// Hi Jane Smith!!
```

ASSIGNMENT – 1 & 2

```
// bind() - Returns new function with bound 'this'
```

```
console.log("\nUsing bind():");
```

```
const getPerson2Name = person1.getFullName.bind(person2);
```

```
console.log(getPerson2Name("Hey", "..."));
```

```
// Hey Jane Smith...
```

```
const getPerson2NameWithHello = person1.getFullName.bind(person2,  
"Hello");
```

```
console.log(getPerson2NameWithHello("!"));
```

```
// Hello Jane Smith!
```

```
// Practical example - Borrowing methods
```

```
const numbers = [1, 2, 3];
```

```
const moreNumbers = [4, 5, 6];
```

```
// Using apply to push array elements individually
```

```
Array.prototype.push.apply(numbers, moreNumbers);
```

```
console.log("\nBorrowing push method:", numbers); // [1, 2, 3, 4, 5, 6]
```

```
// Modern alternative using spread operator
```

```
const arr1 = [1, 2, 3];
```

```
const arr2 = [4, 5, 6];
```

```
arr1.push(...arr2);
```

```
console.log("Using spread operator:", arr1); // [1, 2, 3, 4, 5, 6]
```


ASSIGNMENT – 1 & 2

OBJECT LITERALS

// Program 31: Creating objects with object literals

// Basic object literal

```
const book = {
```

```
  title: "The Great Gatsby",
```

```
  author: "F. Scott Fitzgerald",
```

```
  year: 1925,
```

```
  genre: "Novel",
```

// Method

```
  getDescription: function() {
```

```
    return ` ${this.title} by ${this.author} (${this.year}) `;
```

```
  },
```

// Shorthand method syntax (ES6+)

```
  updateYear(newYear) {
```

```
    this.year = newYear;
```

```
    console.log(`Year updated to ${newYear}`);
```

```
  }
```

```
};
```

```
console.log("Book object:", book);
```

```
console.log("Description:", book.getDescription());
```

ASSIGNMENT – 1 & 2

```
// Accessing properties
```

```
console.log("\nAccessing Properties:");  
console.log("Dot notation:", book.title);  
console.log("Bracket notation:", book["author"]);
```

```
// Adding new properties
```

```
book.publisher = "Scribner";  
book["ISBN"] = "978-0743273565";  
console.log("\nAfter adding properties:", book);
```

```
// Deleting properties
```

```
delete book.genre;  
console.log("\nAfter deleting genre:", book);
```

```
// Checking property existence
```

```
console.log("\nProperty Checks:");  
console.log("Has 'author'?", "author" in book); // true  
console.log("Has 'genre'?", "genre" in book); // false  
console.log("Has 'toString'?", "toString" in book); // true (inherited)  
console.log("Own property 'toString'?", book.hasOwnProperty("toString"));  
// false
```

OBJECT METHODS AND 'THIS' KEYWORD

```
// Program 32: Understanding 'this' in object methods
```

ASSIGNMENT – 1 & 2

```
const user = {  
  name: "Alice",  
  age: 28,  
  
  // Regular method - 'this' refers to the object  
  greet() {  
    console.log(`Hello, I'm ${this.name}`);  
  },  
  
  // Arrow function - 'this' is lexical (from outer scope)  
  greetArrow: () => {  
    console.log(`Hello, I'm ${this.name}`); // undefined!  
  },  
  
  // Method that returns a function  
  getGreeter() {  
    return function() {  
      console.log(`Hello, I'm ${this.name}`); // 'this' will be lost  
    };  
  },  
  
  // Method with preserved 'this'
```

CONSTRUCTOR FUNCTIONS

```
// Program 33: Creating objects with constructor functions
```

ASSIGNMENT – 1 & 2

```
// Constructor function (convention: starts with capital letter)
function Car(make, model, year, color) {
  // Properties
  this.make = make;
  this.model = model;
  this.year = year;
  this.color = color;
  this.isRunning = false;
  this.mileage = 0;

  // Method
  this.start = function() {
    if (!this.isRunning) {
      this.isRunning = true;
      console.log(` ${this.make} ${this.model} started.`);
    } else {
      console.log(` ${this.make} ${this.model} is already running.`);
    }
  };

  this.stop = function() {
    if (this.isRunning) {
      this.isRunning = false;
      console.log(` ${this.make} ${this.model} stopped.`);
    }
  };
}
```

ASSIGNMENT – 1 & 2

```
    } else {  
        console.log(` ${this.make} ${this.model} is already stopped.`);  
    }  
};
```

```
this.drive = function(distance) {  
    if (this.isRunning) {  
        this.mileage += distance;  
        console.log(` ${this.make} ${this.model} drove ${distance} miles.  
Total mileage: ${this.mileage}`);  
    } else {  
        console.log(` Cannot drive. ${this.make} ${this.model} is not  
running.`);  
    }  
};
```

```
this.getInfo = function() {  
    return ` ${this.year} ${this.color} ${this.make} ${this.model} -  
${this.mileage} miles`;  
};  
}
```

// Creating instances with 'new' keyword

```
const car1 = new Car("Toyota", "Camry", 2020, "Silver");  
const car2 = new Car("Honda", "Civic", 2022, "Red");  
const car3 = new Car("Tesla", "Model 3", 2023, "Blue");
```

ASSIGNMENT – 1 & 2

```
console.log("Car 1:", car1.getInfo());  
console.log("Car 2:", car2.getInfo());  
console.log("Car 3:", car3.getInfo());
```

```
console.log("\n--- Testing Car 1 ---");  
car1.start();  
car1.drive(50);  
car1.drive(30);  
car1.stop();  
car1.drive(10); // Won't work - car is stopped
```

```
console.log("\n--- Updated Car 1 Info ---");  
console.log(car1.getInfo());
```

```
// Checking constructor property  
console.log("\n--- Constructor Check ---");  
console.log("car1.constructor === Car:", car1.constructor === Car); //  
true  
console.log("car1 instanceof Car:", car1 instanceof Car); // true  
console.log("car1 instanceof Object:", car1 instanceof Object); // true
```

PROTOTYPES AND PROTOTYPAL INHERITANCE

```
// Program 34: Understanding prototypes and inheritance
```

ASSIGNMENT – 1 & 2

// Constructor with methods defined on prototype (memory efficient)

```
function Animal(name, species) {  
  this.name = name;  
  this.species = species;  
}
```

// Adding methods to prototype (shared across all instances)

```
Animal.prototype.speak = function() {  
  console.log(` ${this.name} makes a sound.`);  
};
```

```
Animal.prototype.eat = function(food) {  
  console.log(` ${this.name} is eating ${food}.`);  
};
```

```
Animal.prototype.sleep = function() {  
  console.log(` ${this.name} is sleeping.`);  
};
```

// Creating instances

```
const animal1 = new Animal("Leo", "Lion");  
const animal2 = new Animal("Milo", "Cat");
```

```
console.log("--- Animal Instances ---");  
animal1.speak();
```

ASSIGNMENT – 1 & 2

```
animal1.eat("meat");
```

```
animal2.speak();
```

```
animal2.sleep();
```

```
// Checking prototype chain
```

```
console.log("\n--- Prototype Chain ---");
```

```
console.log("animal1.__proto__ === Animal.prototype:",  
animal1.__proto__ === Animal.prototype); // true
```

```
console.log("Animal.prototype.__proto__ === Object.prototype:",  
Animal.prototype.__proto__ === Object.prototype); // true
```

```
console.log("Object.prototype.__proto__:", Object.prototype.__proto__); //  
null
```

```
// Inheritance using prototypes
```

```
function Dog(name, breed) {
```

```
    // Call parent constructor
```

```
    Animal.call(this, name, "Dog");
```

```
    this.breed = breed;
```

```
}
```

```
// Set up inheritance chain
```

```
Dog.prototype = Object.create(Animal.prototype);
```

```
Dog.prototype.constructor = Dog;
```

```
// Add methods specific to Dog
```

```
Dog.prototype.bark = function() {
```


ASSIGNMENT – 1 & 2

```
console.log(` ${this.name} says: Woof! Woof!`);  
};  
  
// Override parent method  
Dog.prototype.speak = function() {  
    console.log(` ${this.name} the ${this.breed} barks loudly!`);  
};
```

```
const dog1 = new Dog("Rex", "German Shepherd");  
const dog2 = new Dog("Buddy", "Golden Retriever");
```

```
console.log("\n--- Dog Instances ---");  
dog1.speak(); // Overridden method  
dog1.bark(); // Dog-specific method  
dog1.eat("bones"); // Inherited method  
dog2.sleep(); // Inherited method
```

```
console.log("\n--- Inheritance Check ---");  
console.log("dog1 instanceof Dog:", dog1 instanceof Dog); // true  
console.log("dog1 instanceof Animal:", dog1 instanceof Animal); // true  
console.log("dog1 instanceof Object:", dog1 instanceof Object); // true
```

[ES6 CLASSES - BASIC SYNTAX](#)

// Program 35: Modern class syntax (ES6+)

```
class Person {  
    // Class field (modern syntax)
```

ASSIGNMENT – 1 & 2

```
species = "Homo Sapiens";
```

```
// Constructor
```

```
constructor(firstName, lastName, age) {  
    this.firstName = firstName;  
    this.lastName = lastName;  
    this.age = age;  
    this._email = ""; // Convention for "private" property  
}
```

```
// Instance method
```

```
getFullName() {  
    return `${this.firstName} ${this.lastName}`;  
}
```

```
// Instance method
```

```
introduce() {  
    return `Hi, I'm ${this.getFullName()} and I'm ${this.age} years old.`;  
}
```

```
// Instance method
```

```
celebrateBirthday() {  
    this.age++;  
    console.log(`Happy Birthday ${this.firstName}! You are now  
${this.age} years old.`);
```

ASSIGNMENT – 1 & 2

```
}
```

```
// Getter
```

```
get email() {
```

```
    return this._email;
```

```
}
```

```
// Setter with validation
```

```
set email(value) {
```

```
    if (value.includes("@") && value.includes(".")) {
```

```
        this._email = value;
```

```
    } else {
```

```
        console.log("Invalid email address!");
```

```
    }
```

```
}
```

```
// Static method (called on class, not instance)
```

```
static createAnonymous() {
```

```
    return new Person("Anonymous", "User", 0);
```

```
}
```

```
// Static property
```

```
static speciesCount = 0;
```

```
}
```

ASSIGNMENT – 1 & 2

// Creating instances

```
const person1 = new Person("John", "Doe", 30);
```

```
const person2 = new Person("Jane", "Smith", 25);
```

```
console.log("--- Person Instances ---");
```

```
console.log(person1.introduce());
```

```
console.log(person2.introduce());
```

```
console.log("\n--- Birthday Celebration ---");
```

```
person1.celebrateBirthday();
```

```
person1.celebrateBirthday();
```

```
console.log("\n--- Email Testing ---");
```

```
person1.email = "john.doe@example.com";
```

```
console.log("Valid email set:", person1.email);
```

```
person1.email = "invalid-email";
```

```
console.log("Email after invalid attempt:", person1.email);
```

```
console.log("\n--- Static Methods ---");
```

```
const anonymous = Person.createAnonymous();
```

```
console.log(anonymous.introduce());
```

```
console.log("\n--- Class Fields ---");
```

```
console.log("person1.species:", person1.species);
```

```
console.log("person2.species:", person2.species);
```

ASSIGNMENT – 1 & 2

ES6 CLASSES - INHERITANCE WITH EXTENDS

// Program 36: Class inheritance using extends and super

// Parent class

```
class Vehicle {  
  constructor(make, model, year) {  
    this.make = make;  
    this.model = model;  
    this.year = year;  
    this._speed = 0;  
    this._engine = false;  
  }  
  
  startEngine() {  
    if (!this._engine) {  
      this._engine = true;  
      console.log(` ${this.make} ${this.model} engine started.`);  
    } else {  
      console.log("Engine is already running.");  
    }  
  }  
}  
  
  stopEngine() {  
    if (this._engine) {  
      this._engine = false;
```

ASSIGNMENT – 1 & 2

```
    this._speed = 0;
    console.log(` ${this.make} ${this.model} engine stopped.`);
  } else {
    console.log("Engine is already off.");
  }
}
```

```
accelerate(amount) {
  if (this._engine) {
    this._speed += amount;
    console.log(`Accelerating to ${this._speed} km/h`);
  } else {
    console.log("Start the engine first!");
  }
}
```

```
brake(amount) {
  this._speed = Math.max(0, this._speed - amount);
  console.log(`Braking to ${this._speed} km/h`);
}
```

```
get speed() {
  return this._speed;
}
```

ASSIGNMENT – 1 & 2

```
getInfo() {  
    return `${this.year} ${this.make} ${this.model}`;  
}  
}
```

// Child class inheriting from Vehicle

```
class Car extends Vehicle {  
    constructor(make, model, year, doors, fuelType) {  
        // Call parent constructor  
        super(make, model, year);  
        this.doors = doors;  
        this.fuelType = fuelType;  
        this._trunkOpen = false;  
    }  
}
```

// Additional methods specific to Car

```
openTrunk() {  
    this._trunkOpen = true;  
    console.log("Trunk opened.");  
}
```

```
closeTrunk() {  
    this._trunkOpen = false;  
    console.log("Trunk closed.");  
}
```

ASSIGNMENT – 1 & 2

```
// Override parent method
```

```
getInfo() {  
    return `${super.getInfo()} - ${this.doors} doors, ${this.fuelType}`;  
}
```

```
// Override accelerate with car-specific behavior
```

```
accelerate(amount) {  
    super.accelerate(amount);  
    if (this.speed > 120) {  
        console.log("Warning: High speed detected!");  
    }  
}  
}
```

```
// Another child class
```

```
class Motorcycle extends Vehicle {  
    constructor(make, model, year, type) {  
        super(make, model, year);  
        this.type = type; // Sport, Cruiser, etc.  
        this._helmetOn = false;  
    }
```

```
    putOnHelmet() {  
        this._helmetOn = true;
```


ASSIGNMENT – 1 & 2

```
    console.log("Helmet on - safe to ride!");  
}
```

```
// Override startEngine with safety check
```

```
startEngine() {  
    if (this._helmetOn) {  
        super.startEngine();  
    } else {  
        console.log("Put on your helmet first!");  
    }  
}
```

```
wheelie() {  
    if (this._engine && this._speed > 20) {  
        console.log("🏍️ Doing a wheelie!");  
    } else {  
        console.log("Need more speed for a wheelie!");  
    }  
}
```

```
getInfo() {  
    return `${super.getInfo()} - ${this.type} Motorcycle`;  
}  
}
```

ASSIGNMENT – 1 & 2

```
// Testing the inheritance
```

```
console.log("--- Car Instance ---");
```

```
const myCar = new Car("Toyota", "Camry", 2022, 4, "Hybrid");
```

```
console.log(myCar.getInfo());
```

```
myCar.startEngine();
```

```
myCar.accelerate(50);
```

```
myCar.accelerate(80);
```

```
myCar.brake(30);
```

```
myCar.openTrunk();
```

```
myCar.closeTrunk();
```

```
myCar.stopEngine();
```

```
console.log("\n--- Motorcycle Instance ---");
```

```
const myBike = new Motorcycle("Yamaha", "R6", 2023, "Sport");
```

```
console.log(myBike.getInfo());
```

```
myBike.startEngine(); // Won't start without helmet
```

```
myBike.putOnHelmet();
```

```
myBike.startEngine();
```

```
myBike.accelerate(30);
```

```
myBike.wheelie();
```

```
myBike.stopEngine();
```

```
console.log("\n--- Instance Checks ---");
```

```
console.log("myCar instanceof Car:", myCar instanceof Car); // true
```

```
console.log("myCar instanceof Vehicle:", myCar instanceof Vehicle); // true
```

ASSIGNMENT – 1 & 2

```
console.log("myBike instanceof Motorcycle:", myBike instanceof Motorcycle); // true
```

```
console.log("myBike instanceof Vehicle:", myBike instanceof Vehicle); // true
```

```
console.log("myBike instanceof Car:", myBike instanceof Car); // false
```

OBJECT.CREATE() AND PURE PROTOTYPAL INHERITANCE

// Program 37: Object.create() for prototypal inheritance

// Base object (acts as prototype)

```
const animalProto = {
```

```
  init(name, species) {
```

```
    this.name = name;
```

```
    this.species = species;
```

```
    return this;
```

```
  },
```

```
  speak() {
```

```
    console.log(` ${this.name} makes a sound.`);
```

```
  },
```

```
  eat(food) {
```

```
    console.log(` ${this.name} is eating ${food}.`);
```

```
  },
```

```
  describe() {
```

ASSIGNMENT – 1 & 2

```
    return `${this.name} is a ${this.species}`;  
  }  
};
```

```
// Create objects using Object.create()  
const animal1 = Object.create(animalProto).init("Leo", "Lion");  
const animal2 = Object.create(animalProto).init("Milo", "Cat");
```

```
console.log("--- Objects from animalProto ---");  
console.log(animal1.describe());  
animal1.speak();  
animal1.eat("meat");  
console.log(animal2.describe());  
animal2.speak();
```

```
// Creating a specialized prototype  
const dogProto = Object.create(animalProto);
```

```
// Add methods specific to dogs  
dogProto.bark = function() {  
  console.log(`${this.name} says: Woof!`);  
};
```

```
// Override speak method  
dogProto.speak = function() {
```

ASSIGNMENT – 1 & 2

```
    console.log(` ${this.name} barks loudly!`);  
};  
  
// Custom init for dogs  
dogProto.init = function(name, breed) {  
    animalProto.init.call(this, name, "Dog");  
    this.breed = breed;  
    return this;  
};  
  
dogProto.describe = function() {  
    return ` ${this.name} is a ${this.breed} ${this.species}`;  
};  
  
// Create dog instances  
const dog1 = Object.create(dogProto).init("Rex", "German Shepherd");  
const dog2 = Object.create(dogProto).init("Buddy", "Golden Retriever");  
  
console.log("\n--- Dog Instances ---");  
console.log(dog1.describe());  
dog1.speak();  
dog1.bark();  
dog1.eat("bones");  
  
console.log(dog2.describe());
```

ASSIGNMENT – 1 & 2

```
dog2.speak();
```

```
// Even deeper inheritance chain
```

```
const guardDogProto = Object.create(dogProto);
```

```
guardDogProto.guard = function() {  
    console.log(` ${this.name} is guarding the house!` );  
};
```

```
guardDogProto.init = function(name, breed, trainingLevel) {  
    dogProto.init.call(this, name, breed);  
    this.trainingLevel = trainingLevel;  
    return this;  
};
```

```
const guardDog = Object.create(guardDogProto).init("Max", "Rottweiler",  
"Advanced");
```

```
console.log("\n--- Guard Dog Instance ---");  
console.log(guardDog.describe());  
console.log("Training level:", guardDog.trainingLevel);  
guardDog.speak();  
guardDog.guard();
```

```
// Checking prototype chain
```

ASSIGNMENT – 1 & 2

```
console.log("\n--- Prototype Chain Check ---");
console.log("guardDogProto.isPrototypeOf(guardDog):",
guardDogProto.isPrototypeOf(guardDog)); // true
console.log("dogProto.isPrototypeOf(guardDog):",
dogProto.isPrototypeOf(guardDog)); // true
console.log("animalProto.isPrototypeOf(guardDog):",
animalProto.isPrototypeOf(guardDog)); // true

// Object.create(null) - creates object with no prototype
const dictionary = Object.create(null);
dictionary.hello = "A greeting";
dictionary.world = "The planet Earth";

console.log("\n--- Object without prototype ---");
console.log("dictionary.hello:", dictionary.hello);
console.log("dictionary.toString:", dictionary.toString); // undefined
console.log("'toString' in dictionary:", "toString" in dictionary); // false
```

[PROPERTY DESCRIPTORS AND OBJECT.DEFINEPROPERTY\(\)](#)

// Program 38: Advanced property control with descriptors

```
const user = {};

// Basic property definition
Object.defineProperty(user, 'username', {
  value: 'john_doe',
  writable: true,
```

ASSIGNMENT – 1 & 2

```
enumerable: true,  
configurable: true  
});
```

```
// Read-only property
```

```
Object.defineProperty(user, 'id', {  
  value: 'USR-' + Math.floor(Math.random() * 10000),  
  writable: false,      // Cannot change  
  enumerable: true,     // Shows in loops  
  configurable: false   // Cannot delete or reconfigure  
});
```

```
// Property with getter and setter
```

```
let _age = 0;  
Object.defineProperty(user, 'age', {  
  get() {  
    console.log(`Getting age: ${_age}`);  
    return _age;  
  },  
  set(value) {  
    if (value < 0) {  
      console.log('Age cannot be negative!');  
    } else if (value > 150) {  
      console.log('Age seems unrealistic!');  
    } else {
```


ASSIGNMENT – 1 & 2

```
    console.log(`Setting age to ${value}`);
    _age = value;
  }
},
enumerable: true,
configurable: true
});

// Hidden property (not enumerable)
Object.defineProperty(user, 'password', {
  value: 'secret123',
  writable: true,
  enumerable: false,    // Won't show in for...in or Object.keys()
  configurable: true
});

// Computed property
Object.defineProperty(user, 'profile', {
  get() {
    return `${this.username} (ID: ${this.id}, Age: ${this.age})`;
  },
  enumerable: true,
  configurable: true
});
```

ASSIGNMENT – 1 & 2

```
console.log("--- Testing Properties ---");
user.username = "johndoe_42";
console.log("Username:", user.username);

console.log("\n--- Attempting to modify id ---");
user.id = "new-id";
console.log("ID after modification attempt:", user.id); // Unchanged

console.log("\n--- Testing age setter ---");
user.age = 25; // Valid
user.age = -5; // Invalid
user.age = 200; // Invalid
console.log("Current age:", user.age);

console.log("\n--- Testing computed property ---");
console.log(user.profile);

console.log("\n--- Enumeration ---");
console.log("Object.keys():", Object.keys(user));
console.log("All properties (including non-enumerable):");
console.log("password exists?", user.password);

// Getting property descriptors
console.log("\n--- Property Descriptors ---");
const idDescriptor = Object.getOwnPropertyDescriptor(user, 'id');
```

ASSIGNMENT – 1 & 2

```
console.log("ID Descriptor:", idDescriptor);
```

```
const ageDescriptor = Object.getOwnPropertyDescriptor(user, 'age');
```

```
console.log("Age Descriptor:", ageDescriptor);
```

```
// Defining multiple properties at once
```

```
const product = {};
```

```
Object.defineProperties(product, {
```

```
  name: {
```

```
    value: 'Laptop',
```

```
    writable: true,
```

```
    enumerable: true
```

```
  },
```

```
  price: {
```

```
    value: 999.99,
```

```
    writable: false,
```

```
    enumerable: true
```

```
  },
```

```
  _stock: {
```

```
    value: 50,
```

```
    writable: true,
```

```
    enumerable: false
```

```
  },
```

```
  stock: {
```

```
    get() { return this._stock; },
```

ASSIGNMENT – 1 & 2

```
set(value) {  
    if (value >= 0) this._stock = value;  
},  
enumerable: true  
},  
inStock: {  
    get() { return this._stock > 0; },  
    enumerable: true  
}  
});
```

```
console.log("\n--- Product with Multiple Properties ---");  
console.log("Name:", product.name);  
console.log("Price:", product.price);  
console.log("Stock:", product.stock);  
console.log("In Stock:", product.inStock);
```

```
// Attempting to change read-only price
```

```
product.price = 799.99;
```

```
console.log("Price after modification attempt:", product.price); // Unchanged
```

```
// Sealing and Freezing Objects
```

```
console.log("\n--- Object Sealing and Freezing ---");
```

```
const config = { theme: 'dark', notifications: true };
```

ASSIGNMENT – 1 & 2

```
console.log("Original config:", config);
```

```
Object.seal(config); // Cannot add/delete, but can modify existing
```

```
config.theme = 'light';
```

```
config.newProp = 'test';
```

```
delete config.notifications;
```

```
console.log("After seal - theme changed:", config.theme);
```

```
console.log("After seal - new property added:", config.newProp); //  
undefined
```

```
console.log("After seal - notification deleted:", config.notifications); // still  
true
```

```
const constants = Object.freeze({ PI: 3.14159, MAX_SIZE: 1000 });
```

```
constants.PI = 3.14; // Silently ignored (or error in strict mode)
```

```
console.log("\nFrozen object PI:", constants.PI); // Unchanged
```

[OBJECT METHODS - KEYS, VALUES, ENTRIES, ASSIGN](#)

```
// Program 39: Working with Object static methods
```

```
// Sample object
```

```
const person = {
```

```
  firstName: 'John',
```

```
  lastName: 'Doe',
```

```
  age: 30,
```

```
  email: 'john@example.com',
```

ASSIGNMENT – 1 & 2

isActive: true

};

```
console.log("--- Object.keys() ---");
```

```
const keys = Object.keys(person);
```

```
console.log("Keys:", keys);
```

```
keys.forEach(key => {
```

```
    console.log(` ${key}: ${person[key]} `);
```

```
});
```

```
console.log("\n--- Object.values() ---");
```

```
const values = Object.values(person);
```

```
console.log("Values:", values);
```

```
console.log("Sum of numeric values:",
```

```
    values.filter(v => typeof v === 'number').reduce((a, b) => a + b, 0));
```

```
console.log("\n--- Object.entries() ---");
```

```
const entries = Object.entries(person);
```

```
console.log("Entries:", entries);
```

```
entries.forEach(([key, value]) => {
```

```
    console.log(` ${key} = ${value} `);
```

```
});
```

```
// Convert entries back to object
```

```
const entriesToObject = Object.fromEntries(entries);
```

ASSIGNMENT – 1 & 2

```
console.log("\n--- Object.fromEntries() ---");
console.log("Reconstructed object:", entriesToObject);

// Object.assign() - Merging objects
console.log("\n--- Object.assign() ---");
const defaults = {
  theme: 'light',
  fontSize: 14,
  notifications: true,
  language: 'en'
};

const userSettings = {
  theme: 'dark',
  fontSize: 16
};

const finalSettings = Object.assign({}, defaults, userSettings);
console.log("Default settings:", defaults);
console.log("User settings:", userSettings);
console.log("Merged settings:", finalSettings);

// Deep cloning warning
console.log("\n--- Object.assign() Shallow Copy ---");
const original = {
```

ASSIGNMENT – 1 & 2

```
name: 'Original',
settings: {
  volume: 50,
  brightness: 80
}
};
```

```
const shallowCopy = Object.assign({}, original);
shallowCopy.name = 'Copy';
shallowCopy.settings.volume = 100; // Affects original!
```

```
console.log("Original after modifying copy:", original);
console.log("Shallow copy:", shallowCopy);
console.log("Warning: Nested objects are shared!");
```

```
// Object.hasOwn() - Modern property check
console.log("\n--- Object.hasOwn() ---");
console.log("Has 'firstName':", Object.hasOwn(person, 'firstName')); // true
console.log("Has 'toString':", Object.hasOwn(person, 'toString')); // false
(inherited)
```

```
// Object.is() - Value comparison
console.log("\n--- Object.is() ---");
console.log("Object.is(25, 25):", Object.is(25, 25)); // true
console.log("Object.is(0, -0):", Object.is(0, -0)); // false (unlike ===)
```


ASSIGNMENT – 1 & 2

```
console.log("Object.is(NaN, NaN):", Object.is(NaN, NaN)); // true (unlike ===)
```

```
console.log("0 === -0:", 0 === -0); // true
```

```
console.log("NaN === NaN:", NaN === NaN); // false
```

```
// Object.getOwnPropertyNames() vs Object.keys()
```

```
console.log("\n--- Property Name Methods ---");
```

```
const objWithNonEnum = {};
```

```
Object.defineProperty(objWithNonEnum, 'hidden', {
```

```
  value: 'secret',
```

```
  enumerable: false
```

```
});
```

```
objWithNonEnum.visible = 'public';
```

```
console.log("Object.keys():", Object.keys(objWithNonEnum)); // ['visible']
```

```
console.log("Object.getOwnPropertyNames():",
```

```
Object.getOwnPropertyNames(objWithNonEnum)); // ['hidden', 'visible']
```

```
// Object.groupBy() - Modern grouping (ES2024)
```

```
console.log("\n--- Grouping with Object.groupBy() ---");
```

```
const students = [
```

```
  { name: 'Alice', grade: 'A' },
```

```
  { name: 'Bob', grade: 'B' },
```

```
  { name: 'Charlie', grade: 'A' },
```

```
  { name: 'David', grade: 'C' },
```

ASSIGNMENT – 1 & 2

```
{ name: 'Eve', grade: 'B' }  
];  
  
// If Object.groupBy is available  
if (typeof Object.groupBy === 'function') {  
  const groupedByGrade = Object.groupBy(students, student =>  
    student.grade);  
  console.log("Students grouped by grade:", groupedByGrade);  
} else {  
  // Manual grouping fallback  
  const manualGroup = students.reduce((acc, student) => {  
    if (!acc[student.grade]) acc[student.grade] = [];  
    acc[student.grade].push(student);  
    return acc;  
  }, {});  
  console.log("Students grouped manually:", manualGroup);  
}
```

GETTERS, SETTERS, AND PROPERTY VALIDATION

// Program 40: Advanced getters, setters, and computed properties

```
class BankAccount {  
  #balance = 0;    // Private field (ES2022+)  
  #transactions = []; // Private field  
  #accountNumber;  // Private field
```

ASSIGNMENT – 1 & 2

```
constructor(accountHolder, initialDeposit = 0) {  
    this.accountHolder = accountHolder;  
    this.#accountNumber = this.#generateAccountNumber();  
    this.createdAt = new Date();  
  
    if (initialDeposit > 0) {  
        this.deposit(initialDeposit);  
    }  
}  
  
// Private method  
#generateAccountNumber() {  
    return 'ACC-' + Math.floor(Math.random() *  
1000000).toString().padStart(6, '0');  
}  
  
// Getter for private field (read-only)  
get accountNumber() {  
    return this.#accountNumber;  
}  
  
// Getter for balance (read-only view)  
get balance() {  
    return this.#balance;  
}
```

ASSIGNMENT – 1 & 2

// Getter for formatted balance

```
get formattedBalance() {  
    return new Intl.NumberFormat('en-US', {  
        style: 'currency',  
        currency: 'USD'  
    }).format(this.#balance);  
}
```

// Getter for account status

```
get status() {  
    if (this.#balance < 0) return 'Overdrawn';  
    if (this.#balance === 0) return 'Empty';  
    if (this.#balance < 100) return 'Low Balance';  
    return 'Active';  
}
```

// Getter for recent transactions

```
get recentTransactions() {  
    return this.#transactions.slice(-5).reverse();  
}
```

// Getter for total deposits

```
get totalDeposits() {  
    return this.#transactions
```

ASSIGNMENT – 1 & 2

```
.filter(t => t.type === 'deposit')  
.reduce((sum, t) => sum + t.amount, 0);  
}
```

// Getter for total withdrawals

```
get totalWithdrawals() {  
  return this.#transactions  
    .filter(t => t.type === 'withdrawal')  
    .reduce((sum, t) => sum + t.amount, 0);  
}
```

// Methods for transactions

```
deposit(amount) {  
  if (!this.#validateAmount(amount)) return false;  
  
  this.#balance += amount;  
  this.#addTransaction('deposit', amount);  
  console.log(`Deposited ${this.formatCurrency(amount)}. New balance:  
${this.formattedBalance}`);  
  return true;  
}
```

```
withdraw(amount) {  
  if (!this.#validateAmount(amount)) return false;
```

ASSIGNMENT – 1 & 2

```
if (this.#balance - amount < -500) {  
    console.log('Withdrawal denied: Would exceed overdraft limit of  
$500');  
    return false;  
}  
  
this.#balance -= amount;  
this.#addTransaction('withdrawal', amount);  
console.log(`Withdrew ${this.formatCurrency(amount)}. New balance:  
${this.formattedBalance}`);  
  
if (this.#balance < 0) {  
    console.log('Warning: Account is overdrawn!');  
}  
  
return true;  
}  
  
#validateAmount(amount) {  
    if (typeof amount !== 'number' || isNaN(amount)) {  
        console.log('Error: Amount must be a number');  
        return false;  
    }  
    if (amount <= 0) {  
        console.log('Error: Amount must be positive');
```

ASSIGNMENT – 1 & 2

```
    return false;
  }
  if (amount > 10000) {
    console.log('Error: Amount exceeds transaction limit of $10,000');
    return false;
  }
  return true;
}
```

```
#addTransaction(type, amount) {
  this.#transactions.push({
    type,
    amount,
    date: new Date(),
    balance: this.#balance
  });
}
```

```
formatCurrency(amount) {
  return new Intl.NumberFormat('en-US', {
    style: 'currency',
    currency: 'USD'
  }).format(amount);
}
```

ASSIGNMENT – 1 & 2

```
// Getter for account summary
get summary() {
  return {
    holder: this.accountHolder,
    number: this.accountNumber,
    balance: this.formattedBalance,
    status: this.status,
    created: this.createdAt.toLocaleDateString(),
    totalDeposits: this.formatCurrency(this.totalDeposits),
    totalWithdrawals: this.formatCurrency(this.totalWithdrawals),
    transactionCount: this.#transactions.length
  };
}
}
```

```
// Testing the BankAccount class
console.log("--- Creating Bank Account ---");
const account = new BankAccount("John Doe", 500);
console.log("Account holder:", account.accountHolder);
console.log("Account number:", account.accountNumber);
console.log("Initial balance:", account.formattedBalance);

console.log("\n--- Making Transactions ---");
account.deposit(250);
account.withdraw(100);
```


ASSIGNMENT – 1 & 2

```
account.withdraw(700); // Goes negative
```

```
account.deposit(1000);
```

```
console.log("\n--- Account Status ---");
```

```
console.log("Current balance:", account.formattedBalance);
```

```
console.log("Account status:", account.status);
```

```
console.log("\n--- Recent Transactions ---");
```

```
account.recentTransactions.forEach(t => {
```

```
    console.log(` ${t.date.toLocaleTimeString()}: ${t.type} of  
    ${account.formatCurrency(t.amount)} `);
```

```
});
```

```
console.log("\n--- Account Summary ---");
```

```
const summary = account.summary;
```

```
Object.entries(summary).forEach(([key, value]) => {
```

```
    console.log(` ${key}: ${value} `);
```

```
});
```

```
// Testing validation
```

```
console.log("\n--- Testing Validation ---");
```

```
account.deposit(-50); // Invalid
```

```
account.deposit(15000); // Exceeds limit
```

```
account.withdraw("100"); // Not a number
```

ASSIGNMENT – 1 & 2

// Another class with computed getters/setters

```
class Temperature {
```

```
  #celsius = 0;
```

```
  constructor(value, unit = 'C') {
```

```
    this.setTemperature(value, unit);
```

```
  }
```

```
  setTemperature(value, unit) {
```

```
    if (typeof value !== 'number' || isNaN(value)) {
```

```
      throw new Error('Temperature must be a number');
```

```
    }
```

```
    switch(unit.toUpperCase()) {
```

```
      case 'C':
```

```
        this.#celsius = value;
```

```
        break;
```

```
      case 'F':
```

```
        this.#celsius = (value - 32) * 5/9;
```

```
        break;
```

```
      case 'K':
```

```
        this.#celsius = value - 273.15;
```

```
        break;
```

```
      default:
```

```
        throw new Error('Unknown temperature unit');
```

ASSIGNMENT – 1 & 2

```
    }  
}  
  
// Getters for different units  
get celsius() {  
    return this.#celsius;  
}  
  
get fahrenheit() {  
    return this.#celsius * 9/5 + 32;  
}  
  
get kelvin() {  
    return this.#celsius + 273.15;  
}  
  
// Setters for different units  
set celsius(value) {  
    this.setTemperature(value, 'C');  
}  
  
set fahrenheit(value) {  
    this.setTemperature(value, 'F');  
}
```

ASSIGNMENT – 1 & 2

```
set kelvin(value) {  
  this.setTemperature(value, 'K');  
}
```

```
// Computed property
```

```
get description() {  
  if (this.#celsius < 0) return 'Freezing';  
  if (this.#celsius < 10) return 'Cold';  
  if (this.#celsius < 20) return 'Cool';  
  if (this.#celsius < 30) return 'Warm';  
  return 'Hot';  
}
```

```
// String representation
```

```
toString() {  
  return `${this.#celsius.toFixed(1)}°C / ${this.fahrenheit.toFixed(1)}°F /  
  ${this.kelvin.toFixed(1)}K (${this.description})`;  
}  
}
```

```
console.log("\n--- Temperature Converter ---");  
const temp = new Temperature(25, 'C');  
console.log("Initial:", temp.toString());  
  
temp.fahrenheit = 98.6;
```

ASSIGNMENT – 1 & 2

```
console.log("After setting to 98.6°F:", temp.toString());
```

```
temp.kelvin = 300;
```

```
console.log("After setting to 300K:", temp.toString());
```

ARRAY CREATION AND BASIC METHODS

```
// Program 41: Creating and manipulating arrays
```

```
// Array creation methods
```

```
const arr1 = [1, 2, 3, 4, 5];           // Array literal
```

```
const arr2 = new Array(1, 2, 3, 4, 5);   // Array constructor
```

```
const arr3 = Array.from('hello');        // From iterable
```

```
const arr4 = Array.of(1, 2, 3, 4, 5);    // Array.of()
```

```
console.log("--- Array Creation ---");
```

```
console.log("Array literal:", arr1);
```

```
console.log("Array constructor:", arr2);
```

```
console.log("Array.from('hello'):", arr3);
```

```
console.log("Array.of():", arr4);
```

```
// Length property and sparse arrays
```

```
console.log("\n--- Array Length ---");
```

```
const arr5 = [1, 2, 3];
```

```
console.log("Length:", arr5.length);
```

```
arr5.length = 5;
```

ASSIGNMENT – 1 & 2

```
console.log("After extending length:", arr5); // [1, 2, 3, empty × 2]
```

```
arr5.length = 2;
```

```
console.log("After truncating length:", arr5); // [1, 2]
```

```
// Basic array methods
```

```
console.log("\n--- Basic Methods ---");
```

```
const fruits = ['apple', 'banana', 'orange'];
```

```
console.log("Original:", fruits);
```

```
console.log("indexOf('banana'):", fruits.indexOf('banana')); // 1
```

```
console.log("indexOf('grape'):", fruits.indexOf('grape')); // -1
```

```
console.log("includes('orange'):", fruits.includes('orange')); // true
```

```
console.log("includes('grape'):", fruits.includes('grape')); // false
```

```
// Join method
```

```
console.log("\n--- Join ---");
```

```
console.log("fruits.join():", fruits.join()); // "apple,banana,orange"
```

```
console.log("fruits.join(' | '):", fruits.join(' | ')); // "apple | banana | orange"
```

```
console.log("fruits.join(''):", fruits.join('')); // "applebananaorange"
```

```
// Concat method
```

```
console.log("\n--- Concat ---");
```

```
const moreFruits = ['grape', 'kiwi'];
```

```
const allFruits = fruits.concat(moreFruits);
```

```
console.log("Concatenated:", allFruits);
```

ASSIGNMENT – 1 & 2

```
console.log("Multiple concat:", fruits.concat(moreFruits, ['mango', 'peach']));
```

```
// Slice method (doesn't modify original)
```

```
console.log("\n--- Slice ---");
```

```
const numbers = [10, 20, 30, 40, 50];
```

```
console.log("Original:", numbers);
```

```
console.log("slice(1, 3):", numbers.slice(1, 3)); // [20, 30]
```

```
console.log("slice(2):", numbers.slice(2));      // [30, 40, 50]
```

```
console.log("slice(-2):", numbers.slice(-2));    // [40, 50]
```

```
console.log("slice():", numbers.slice());        // Shallow copy
```

```
// Check if array
```

```
console.log("\n--- Array Checking ---");
```

```
console.log("Array.isArray([]):", Array.isArray([]));      // true
```

```
console.log("Array.isArray({}):", Array.isArray({}));      // false
```

```
console.log("Array.isArray('hello'):", Array.isArray('hello')); // false
```

ARRAY ITERATION - FOREACH, MAP, FILTER

```
// Program 42: Array iteration methods
```

```
const numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
```

```
const students = [
```

```
  { name: 'Alice', grade: 85, major: 'CS' },
```

```
  { name: 'Bob', grade: 92, major: 'Math' },
```

```
  { name: 'Charlie', grade: 78, major: 'CS' },
```

```
  { name: 'David', grade: 95, major: 'Physics' },
```

ASSIGNMENT – 1 & 2

```
{ name: 'Eve', grade: 88, major: 'CS' }  
];  
  
console.log("--- forEach (side effects only) ---");  
console.log("Doubling each number:");  
numbers.forEach((num, index) => {  
    console.log(`Index ${index}: ${num} × 2 = ${num * 2}`);  
});  
  
console.log("\n--- map (transformation) ---");  
const doubled = numbers.map(num => num * 2);  
console.log("Original:", numbers);  
console.log("Doubled:", doubled);  
  
const studentNames = students.map(student => student.name);  
console.log("Student names:", studentNames);  
  
const studentSummaries = students.map(s => ({  
    name: s.name,  
    passed: s.grade >= 80,  
    grade: s.grade  
}));  
console.log("Student summaries:", studentSummaries);  
  
console.log("\n--- filter (selection) ---");
```


ASSIGNMENT – 1 & 2

```
const evenNumbers = numbers.filter(num => num % 2 === 0);  
console.log("Even numbers:", evenNumbers);
```

```
const csStudents = students.filter(s => s.major === 'CS');  
console.log("CS Students:", csStudents);
```

```
const highAchievers = students.filter(s => s.grade >= 90);  
console.log("Students with grade >= 90:", highAchievers);
```

```
console.log("\n--- Chaining Methods ---");  
const averageCSGrade = students  
  .filter(s => s.major === 'CS')  
  .map(s => s.grade)  
  .reduce((sum, grade, _, arr) => sum + grade / arr.length, 0);  
console.log("Average CS grade:", averageCSGrade.toFixed(2));
```

```
console.log("\n--- Performance Comparison ---");  
const largeArray = Array.from({ length: 1000 }, (_, i) => i);
```

```
console.time('forEach');  
largeArray.forEach(num => { const x = num * 2; });  
console.timeEnd('forEach');
```

```
console.time('for loop');  
for (let i = 0; i < largeArray.length; i++) {
```

ASSIGNMENT – 1 & 2

```
const x = largeArray[i] * 2;  
}
```

```
console.timeEnd('for loop');
```

ARRAY TRANSFORMATION - REDUCE, REDUCERIGHT

```
// Program 43: Array reduction methods
```

```
const numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
```

```
const transactions = [  
  { type: 'deposit', amount: 500 },  
  { type: 'withdrawal', amount: 200 },  
  { type: 'deposit', amount: 300 },  
  { type: 'withdrawal', amount: 100 },  
  { type: 'deposit', amount: 150 }  
];
```

```
console.log("--- Basic reduce ---");
```

```
const sum = numbers.reduce((accumulator, current) => accumulator +  
current, 0);
```

```
console.log("Sum of numbers:", sum);
```

```
const product = numbers.reduce((acc, cur) => acc * cur, 1);
```

```
console.log("Product of numbers:", product);
```

```
console.log("\n--- reduce without initial value ---");
```

```
const max = numbers.reduce((acc, cur) => cur > acc ? cur : acc);
```

ASSIGNMENT – 1 & 2

```
console.log("Maximum value:", max);
```

```
const min = numbers.reduce((acc, cur) => cur < acc ? cur : acc);
```

```
console.log("Minimum value:", min);
```

```
console.log("\n--- reduce with objects ---");
```

```
const balance = transactions.reduce((acc, transaction) => {
```

```
  if (transaction.type === 'deposit') {
```

```
    return acc + transaction.amount;
```

```
  } else {
```

```
    return acc - transaction.amount;
```

```
  }
```

```
}, 0);
```

```
console.log("Final balance:", balance);
```

```
console.log("\n--- reduce for grouping ---");
```

```
const groupedByType = transactions.reduce((acc, t) => {
```

```
  if (!acc[t.type]) acc[t.type] = [];
```

```
  acc[t.type].push(t.amount);
```

```
  return acc;
```

```
}, {});
```

```
console.log("Grouped by type:", groupedByType);
```

```
console.log("\n--- reduce for counting ---");
```

```
const items = ['apple', 'banana', 'apple', 'orange', 'banana', 'apple'];
```

ASSIGNMENT – 1 & 2

```
const itemCounts = items.reduce((acc, item) => {  
  acc[item] = (acc[item] || 0) + 1;  
  return acc;  
}, {});  
console.log("Item counts:", itemCounts);
```

```
console.log("\n--- reduce for flattening ---");  
const nested = [[1, 2], [3, 4], [5, 6]];  
const flattened = nested.reduce((acc, curr) => acc.concat(curr), []);  
console.log("Flattened array:", flattened);
```

```
console.log("\n--- reduceRight (right-to-left) ---");  
const words = ['World', 'Hello'];  
const sentence = words.reduceRight((acc, word) => acc + ' ' + word);  
console.log("reduceRight result:", sentence);
```

```
console.log("\n--- Complex reduce example: Pipeline ---");  
const pipeline = [  
  (x) => x + 10,  
  (x) => x * 2,  
  (x) => x - 5,  
  (x) => x / 3  
];
```

```
const result = pipeline.reduce((value, fn) => fn(value), 5);
```

ASSIGNMENT – 1 & 2

```
console.log("Pipeline result for 5:", result);
```

ARRAY SEARCH - FIND, FINDINDEX, SOME, EVERY

```
// Program 44: Array search and testing methods
```

```
const users = [
```

```
  { id: 1, name: 'Alice', age: 25, active: true },
```

```
  { id: 2, name: 'Bob', age: 30, active: false },
```

```
  { id: 3, name: 'Charlie', age: 35, active: true },
```

```
  { id: 4, name: 'David', age: 28, active: true },
```

```
  { id: 5, name: 'Eve', age: 22, active: false }
```

```
];
```

```
const numbers = [10, 25, 30, 45, 60, 75, 80];
```

```
console.log("--- find() - returns first matching element ---");
```

```
const userOver30 = users.find(user => user.age > 30);
```

```
console.log("First user over 30:", userOver30);
```

```
const inactiveUser = users.find(u => !u.active);
```

```
console.log("First inactive user:", inactiveUser);
```

```
const notFound = users.find(u => u.age > 100);
```

```
console.log("Not found result:", notFound);
```

```
console.log("\n--- findIndex() - returns index of first match ---");
```

ASSIGNMENT – 1 & 2

```
const indexOver30 = users.findIndex(user => user.age > 30);  
console.log("Index of first user over 30:", indexOver30);
```

```
const indexNotFound = users.findIndex(u => u.age > 100);  
console.log("Index when not found:", indexNotFound);
```

```
console.log("\n--- findLast() and findLastIndex() (ES2023+) ---");  
if (typeof Array.prototype.findLast === 'function') {  
  const lastActive = users.findLast(u => u.active);  
  console.log("Last active user:", lastActive);  
  console.log("Last active index:", users.findLastIndex(u => u.active));  
}
```

```
console.log("\n--- some() - checks if ANY element matches ---");  
const hasInactiveUsers = users.some(user => !user.active);  
console.log("Has inactive users:", hasInactiveUsers);
```

```
const hasSeniorCitizen = users.some(user => user.age >= 65);  
console.log("Has senior citizen:", hasSeniorCitizen);
```

```
const hasLargeNumber = numbers.some(num => num > 100);  
console.log("Has number > 100:", hasLargeNumber);
```

```
console.log("\n--- every() - checks if ALL elements match ---");  
const allActive = users.every(user => user.active);
```

ASSIGNMENT – 1 & 2

```
console.log("All users active:", allActive);
```

```
const allAdults = users.every(user => user.age >= 18);
```

```
console.log("All users are adults:", allAdults);
```

```
const allPositive = numbers.every(num => num > 0);
```

```
console.log("All numbers positive:", allPositive);
```

```
console.log("\n--- includes() - simple value check ---");
```

```
const fruits = ['apple', 'banana', 'orange', 'mango'];
```

```
console.log("Includes 'banana':", fruits.includes('banana'));
```

```
console.log("Includes 'grape':", fruits.includes('grape'));
```

```
console.log("Includes 'banana' from index 2:", fruits.includes('banana', 2));
```

```
console.log("\n--- Combining search methods ---");
```

```
function findUserByProperty(users, property, value) {
```

```
    return users.find(user => user[property] === value);
```

```
}
```

```
console.log("User with id 3:", findUserByProperty(users, 'id', 3));
```

```
console.log("User named 'Eve':", findUserByProperty(users, 'name', 'Eve'));
```

```
console.log("\n--- Practical example: Form validation ---");
```

```
const formFields = [
```

```
    { name: 'username', value: 'john_doe', required: true },
```

ASSIGNMENT – 1 & 2

```
{ name: 'email', value: 'john@example.com', required: true },  
{ name: 'age', value: 25, required: true },  
{ name: 'phone', value: '', required: false }  
];
```

```
const allRequiredFilled = formFields  
  .filter(field => field.required)  
  .every(field => field.value !== '');  
console.log("All required fields filled:", allRequiredFilled);
```

```
const firstEmptyRequired = formFields.find(field => field.required &&  
!field.value);  
console.log("First empty required field:", firstEmptyRequired?.name ||  
'none');
```

ARRAY MODIFICATION - PUSH, POP, SHIFT, UNSHIFT, SPLICE

// Program 45: Methods that modify arrays

```
let fruits = ['apple', 'banana', 'orange'];  
console.log("Original array:", fruits);
```

```
console.log("\n--- push() - add to end ---");  
let pushResult = fruits.push('mango', 'grape');  
console.log("After push:", fruits);  
console.log("push() return value (new length):", pushResult);
```


ASSIGNMENT – 1 & 2

```
console.log("\n--- pop() - remove from end ---");  
let popped = fruits.pop();  
console.log("After pop():", fruits);  
console.log("pop() return value:", popped);
```

```
console.log("\n--- unshift() - add to beginning ---");  
let unshiftResult = fruits.unshift('kiwi', 'peach');  
console.log("After unshift():", fruits);  
console.log("unshift() return value:", unshiftResult);
```

```
console.log("\n--- shift() - remove from beginning ---");  
let shifted = fruits.shift();  
console.log("After shift():", fruits);  
console.log("shift() return value:", shifted);
```

```
console.log("\n--- splice() - Swiss Army knife ---");  
let colors = ['red', 'green', 'blue', 'yellow', 'purple'];  
console.log("Colors array:", colors);
```

```
// Remove elements
```

```
let removed = colors.splice(1, 2);  
console.log("\nAfter splice(1, 2) - remove 2 from index 1:");  
console.log("Modified array:", colors);  
console.log("Removed elements:", removed);
```

ASSIGNMENT – 1 & 2

// Insert elements

```
colors = ['red', 'green', 'blue'];  
console.log("\nBefore insert:", colors);  
colors.splice(1, 0, 'yellow', 'orange');  
console.log("After splice(1, 0, 'yellow', 'orange'):", colors);
```

// Replace elements

```
colors = ['red', 'green', 'blue'];  
console.log("\nBefore replace:", colors);  
let replaced = colors.splice(1, 1, 'cyan', 'magenta');  
console.log("After splice(1, 1, 'cyan', 'magenta'):", colors);  
console.log("Replaced element:", replaced);
```

console.log("\n--- splice() with negative index ---");

```
let letters = ['a', 'b', 'c', 'd', 'e'];  
console.log("Letters:", letters);  
let removedFromEnd = letters.splice(-2, 1);  
console.log("After splice(-2, 1):", letters);  
console.log("Removed:", removedFromEnd);
```

console.log("\n--- fill() - fill array with value ---");

```
let numbers = [1, 2, 3, 4, 5];  
console.log("Original:", numbers);  
numbers.fill(0);  
console.log("After fill(0):", numbers);
```

ASSIGNMENT – 1 & 2

```
numbers = [1, 2, 3, 4, 5];  
numbers.fill(9, 1, 4);  
console.log("After fill(9, 1, 4):", numbers);
```

```
console.log("\n--- copyWithin() - copy within array ---");  
let arr = [1, 2, 3, 4, 5];  
console.log("Original:", arr);  
arr.copyWithin(0, 3);  
console.log("After copyWithin(0, 3):", arr);
```

```
arr = [1, 2, 3, 4, 5];  
arr.copyWithin(1, 2, 4);  
console.log("After copyWithin(1, 2, 4):", arr);
```

```
console.log("\n--- Practical Stack and Queue Examples ---");  
// Stack (LIFO) using push/pop  
let stack = [];  
stack.push('page1');  
stack.push('page2');  
stack.push('page3');  
console.log("Stack:", stack);  
console.log("Pop:", stack.pop());  
console.log("Pop:", stack.pop());  
console.log("Stack after pops:", stack);
```

ASSIGNMENT – 1 & 2

```
// Queue (FIFO) using push/shift
let queue = [];
queue.push('task1');
queue.push('task2');
queue.push('task3');
console.log("\nQueue:", queue);
console.log("Dequeue:", queue.shift());
console.log("Dequeue:", queue.shift());
console.log("Queue after dequeues:", queue);
```

ARRAY SORTING AND REVERSING

```
// Program 46: Sorting arrays with custom comparators
```

```
console.log("--- Default sort (lexicographic) ---");
let fruits = ['banana', 'Apple', 'cherry', 'date'];
console.log("Original:", fruits);
console.log("Default sort:", fruits.sort());
console.log("Original modified!", fruits);
```

```
console.log("\n--- Numeric sort (with compare function) ---");
let numbers = [10, 5, 40, 25, 100, 1, 3];
console.log("Original:", numbers);
```

```
// Ascending
let ascending = [...numbers].sort((a, b) => a - b);
```

ASSIGNMENT – 1 & 2

```
console.log("Ascending:", ascending);
```

```
// Descending
```

```
let descending = [...numbers].sort((a, b) => b - a);
```

```
console.log("Descending:", descending);
```

```
console.log("\n--- Sorting objects ---");
```

```
const students = [
```

```
  { name: 'Charlie', grade: 85, age: 20 },
```

```
  { name: 'Alice', grade: 92, age: 22 },
```

```
  { name: 'Bob', grade: 78, age: 19 },
```

```
  { name: 'David', grade: 95, age: 21 },
```

```
  { name: 'Eve', grade: 88, age: 20 }
```

```
];
```

```
// Sort by grade (ascending)
```

```
const byGrade = [...students].sort((a, b) => a.grade - b.grade);
```

```
console.log("Sorted by grade:", byGrade.map(s =>  
  `${s.name}:${s.grade}`));
```

```
// Sort by name (alphabetical)
```

```
const byName = [...students].sort((a, b) =>  
  a.name.localeCompare(b.name));
```

```
console.log("Sorted by name:", byName.map(s => s.name));
```

ASSIGNMENT – 1 & 2

```
// Sort by age descending, then by grade descending
const byAgeThenGrade = [...students].sort((a, b) => {
  if (a.age !== b.age) return b.age - a.age;
  return b.grade - a.grade;
});

console.log("Sorted by age desc, then grade desc:",
  byAgeThenGrade.map(s => `${s.name} (${s.age}, ${s.grade})`));

console.log("\n--- reverse() ---");
let arr = [1, 2, 3, 4, 5];
console.log("Original:", arr);
arr.reverse();
console.log("After reverse():", arr);

console.log("\n--- toSorted() and toReversed() (ES2023+) ---");
if (typeof Array.prototype.toSorted === 'function') {
  let original = [3, 1, 4, 1, 5];
  let sorted = original.toSorted((a, b) => a - b);
  console.log("Original unchanged:", original);
  console.log("New sorted array:", sorted);

  let reversed = original.toReversed();
  console.log("New reversed array:", reversed);
}
```

ASSIGNMENT – 1 & 2

```
console.log("\n--- Complex sorting example ---");

const tasks = [
  { name: 'Task A', priority: 'high', completed: false },
  { name: 'Task B', priority: 'low', completed: true },
  { name: 'Task C', priority: 'medium', completed: false },
  { name: 'Task D', priority: 'high', completed: true },
  { name: 'Task E', priority: 'low', completed: false }
];

const priorityOrder = { 'high': 1, 'medium': 2, 'low': 3 };

const sortedTasks = [...tasks].sort((a, b) => {
  // Incomplete tasks first
  if (a.completed !== b.completed) return a.completed ? 1 : -1;
  // Then by priority
  return priorityOrder[a.priority] - priorityOrder[b.priority];
});

console.log("Sorted tasks (incomplete first, then priority):");
sortedTasks.forEach(t => {
  console.log(` ${t.completed ? '✓' : '○'} ${t.name} (${t.priority})`);
});

console.log("\n--- Stable sorting ---");
const items = [
```

ASSIGNMENT – 1 & 2

```
{ category: 'A', value: 1 },  
{ category: 'B', value: 2 },  
{ category: 'A', value: 3 },  
{ category: 'B', value: 4 }  
];
```

```
const sorted = items.sort((a, b) => a.category.localeCompare(b.category));  
console.log("Stable sort by category:", sorted);
```

MULTIDIMENSIONAL ARRAYS

// Program 47: Working with 2D and 3D arrays

```
console.log("--- Creating 2D Arrays ---");  
const matrix = [  
  [1, 2, 3],  
  [4, 5, 6],  
  [7, 8, 9]  
];
```

```
console.log("Matrix:");  
matrix.forEach(row => console.log(row));
```

```
console.log("\n--- Accessing elements ---");  
console.log("matrix[0][0]:", matrix[0][0]); // 1  
console.log("matrix[1][2]:", matrix[1][2]); // 6  
console.log("matrix[2][1]:", matrix[2][1]); // 8
```


ASSIGNMENT – 1 & 2

```
console.log("\n--- Iterating 2D array ---");
console.log("Row by row:");
for (let i = 0; i < matrix.length; i++) {
  for (let j = 0; j < matrix[i].length; j++) {
    console.log(`matrix[${i}][${j}] = ${matrix[i][j]}`);
  }
}
```

```
console.log("\n--- Creating dynamic 2D array ---");
function create2DArray(rows, cols, initialValue = 0) {
  return Array.from({ length: rows },
    () => Array.from({ length: cols }, () => initialValue));
}
```

```
const grid = create2DArray(3, 4, '.');
console.log("3x4 grid:");
grid.forEach(row => console.log(row.join(' ')));
```

```
console.log("\n--- Tic-Tac-Toe Board ---");
let board = create2DArray(3, 3, ' ');
board[0][0] = 'X';
board[1][1] = 'O';
board[2][2] = 'X';
```

ASSIGNMENT – 1 & 2

```
console.log("Tic-Tac-Toe:");
console.log(' ' + board[0].join(' | '));
console.log('---+---+---');
console.log(' ' + board[1].join(' | '));
console.log('---+---+---');
console.log(' ' + board[2].join(' | '));
```

```
console.log("\n--- Matrix Operations ---");
function transpose(matrix) {
  return matrix[0].map((_, colIndex) =>
    matrix.map(row => row[colIndex])
  );
}
```

```
console.log("Original matrix:");
matrix.forEach(row => console.log(row));
console.log("\nTransposed:");
const transposed = transpose(matrix);
transposed.forEach(row => console.log(row));
```

```
console.log("\n--- 3D Array Example ---");
const cube = [
  [ // Layer 0
    [1, 2],
    [3, 4]
```

ASSIGNMENT – 1 & 2

```
],  
  [ // Layer 1  
    [5, 6],  
    [7, 8]  
  ]  
];  
  
console.log("3D Array (2x2x2):");  
console.log("cube[0][0][0]:", cube[0][0][0]); // 1  
console.log("cube[1][1][1]:", cube[1][1][1]); // 8  
  
console.log("\n--- Practical: Grade Book ---");  
const gradeBook = [  
  ['Student', 'Math', 'Science', 'English', 'Average'],  
  ['Alice', 85, 92, 88],  
  ['Bob', 78, 85, 90],  
  ['Charlie', 92, 88, 85],  
  ['David', 88, 90, 92]  
];  
  
// Calculate averages  
for (let i = 1; i < gradeBook.length; i++) {  
  const scores = gradeBook[i].slice(1);  
  const avg = scores.reduce((a, b) => a + b, 0) / scores.length;  
  gradeBook[i].push(avg.toFixed(1));  
}
```

ASSIGNMENT – 1 & 2

```
}
```

```
// Display grade book
```

```
console.log("\nGrade Book:");  
gradeBook.forEach(row => {  
    console.log(row.map(cell =>  
        cell.toString().padStart(10, ' ')  
    ).join(""));  
});
```

```
console.log("\n--- Flattening arrays ---");  
const nested = [[1, 2], [3, 4, [5, 6]], [7, 8]];  
console.log("Nested:", nested);  
console.log("flat():", nested.flat());  
console.log("flat(2):", nested.flat(2));  
console.log("flat(Infinity):", nested.flat(Infinity));
```

ARRAY SPREAD OPERATOR AND DESTRUCTURING

```
// Program 48: Spread operator and destructuring with arrays
```

```
console.log("--- Spread Operator (...) ---");
```

```
// Copy array
```

```
const original = [1, 2, 3];  
const copy = [...original];  
console.log("Original:", original);
```

ASSIGNMENT – 1 & 2

```
console.log("Copy:", copy);  
console.log("copy === original:", copy === original); // false
```

```
// Merge arrays
```

```
const arr1 = [1, 2, 3];  
const arr2 = [4, 5, 6];  
const arr3 = [7, 8, 9];  
const merged = [...arr1, ...arr2, ...arr3];  
console.log("\nMerged arrays:", merged);
```

```
// Insert elements
```

```
const numbers = [1, 5, 6];  
const withInsert = [0, ...numbers, 7, 8];  
console.log("\nWith insertions:", withInsert);
```

```
// Convert iterables to array
```

```
const str = "Hello";  
const chars = [...str];  
console.log("\nString to array:", chars);
```

```
// Spread in function arguments
```

```
function sum(a, b, c) {  
    return a + b + c;  
}
```

```
const nums = [10, 20, 30];
```

ASSIGNMENT – 1 & 2

```
console.log("\nsum(...nums):", sum(...nums));
```

```
// Math operations with spread
```

```
const values = [5, 2, 9, 1, 7];
```

```
console.log("\nMath.max(...values):", Math.max(...values));
```

```
console.log("Math.min(...values):", Math.min(...values));
```

```
console.log("\n--- Array Destructuring ---");
```

```
// Basic destructuring
```

```
const colors = ['red', 'green', 'blue'];
```

```
const [first, second, third] = colors;
```

```
console.log("Destructured:", first, second, third);
```

```
// Skip elements
```

```
const [primary, , tertiary] = colors;
```

```
console.log("Skipped second:", primary, tertiary);
```

```
// Default values
```

```
const smallArray = ['only one'];
```

```
const [a = 'default1', b = 'default2'] = smallArray;
```

```
console.log("With defaults:", a, b);
```

```
// Rest pattern
```

```
const [head, ...tail] = [1, 2, 3, 4, 5];
```

ASSIGNMENT – 1 & 2

```
console.log("\nHead:", head);
```

```
console.log("Tail:", tail);
```

```
// Swapping variables
```

```
let x = 10, y = 20;
```

```
console.log("\nBefore swap: x =", x, "y =", y);
```

```
[x, y] = [y, x];
```

```
console.log("After swap: x =", x, "y =", y);
```

```
// Nested destructuring
```

```
const nestedArray = [1, [2, 3], 4];
```

```
const [firstNum, [secondNum, thirdNum], fourthNum] = nestedArray;
```

```
console.log("\nNested destructuring:", firstNum, secondNum, thirdNum,  
fourthNum);
```

```
// Destructuring in function parameters
```

```
function processCoords([x, y]) {
```

```
    return `X: ${x}, Y: ${y}`;
```

```
}
```

```
console.log("\nprocessCoords([5, 10]):", processCoords([5, 10]));
```

```
// Returning multiple values
```

```
function getMinMax(arr) {
```

```
    return [Math.min(...arr), Math.max(...arr)];
```

```
}
```

ASSIGNMENT – 1 & 2

```
const [min, max] = getMinMax([3, 7, 1, 9, 4]);
```

```
console.log("\nMin:", min, "Max:", max);
```

```
console.log("\n--- Practical: CSV Parsing ---");
```

```
const csvLine = 'John,Doe,30,Engineer';
```

```
const [firstName, lastName, age, profession] = csvLine.split(',');
```

```
console.log("Parsed CSV:", { firstName, lastName, age, profession });
```

```
console.log("\n--- Practical: Function with named array parameters ---");
```

```
function calculateStats([...numbers]) {
```

```
    const sum = numbers.reduce((a, b) => a + b, 0);
```

```
    const avg = sum / numbers.length;
```

```
    const min = Math.min(...numbers);
```

```
    const max = Math.max(...numbers);
```

```
    return { sum, avg, min, max };
```

```
}
```

```
const stats = calculateStats([85, 92, 78, 95, 88]);
```

```
console.log("Stats:", stats);
```

[ARRAY.FROM\(\) AND ARRAY.OF\(\)](#)

```
// Program 49: Array creation utility methods
```

```
console.log("--- Array.from() ---");
```

```
// From string
```

```
const fromString = Array.from('JavaScript');
```

```
console.log("From string:", fromString);
```


ASSIGNMENT – 1 & 2

```
// From Set
const set = new Set([1, 2, 3, 2, 1, 4]);
const fromSet = Array.from(set);
console.log("From Set (deduplicated):", fromSet);

// From Map
const map = new Map([
  ['a', 1],
  ['b', 2],
  ['c', 3]
]);
const fromMap = Array.from(map);
console.log("From Map:", fromMap);
const mapKeys = Array.from(map.keys());
const mapValues = Array.from(map.values());
console.log("Map keys:", mapKeys);
console.log("Map values:", mapValues);

// From NodeList (simulated)
const divs = { 0: 'div1', 1: 'div2', 2: 'div3', length: 3 };
const fromArrayLike = Array.from(divs);
console.log("From array-like object:", fromArrayLike);

// From arguments object
function argsToArray() {
  return Array.from(arguments);
}
```

ASSIGNMENT – 1 & 2

```
}  
console.log("From arguments:", argsToArray('a', 'b', 'c', 'd'));  
  
console.log("\n--- Array.from() with mapping function ---");  
// Create range with mapping  
const squares = Array.from({ length: 5 }, (_, i) => (i + 1) ** 2);  
console.log("Squares of 1-5:", squares);  
  
// Generate random numbers  
const randomNumbers = Array.from({ length: 5 }, () =>  
Math.floor(Math.random() * 100));  
console.log("Random numbers:", randomNumbers);  
// Create sequence  
const sequence = Array.from({ length: 10 }, (_, i) => i * 2);  
console.log("Even numbers 0-18:", sequence);  
// Generate alphabet  
const alphabet = Array.from({ length: 26 }, (_, i) =>  
String.fromCharCode(65 + i));  
console.log("Alphabet:", alphabet);  
console.log("\n--- Array.of() ---");  
  
// Creates array from arguments (unlike Array constructor)  
const of1 = Array.of(7);  
console.log("Array.of(7):", of1); // [7]  
console.log("new Array(7):", new Array(7)); // [empty × 7]
```

ASSIGNMENT – 1 & 2

```
const of2 = Array.of(1, 2, 3, 4, 5);  
console.log("Array.of(1, 2, 3, 4, 5):", of2);
```

```
const of3 = Array.of('hello', true, 42, { name: 'obj' });  
console.log("Mixed types:", of3);
```

```
console.log("\n--- Practical: Creating matrix with Array.from() ---");  
function createMatrix(rows, cols, initialFn = (r, c) => 0) {  
  return Array.from({ length: rows }, (_, r) =>  
    Array.from({ length: cols }, (_, c) => initialFn(r, c))  
  );  
}
```

```
const identityMatrix = createMatrix(3, 3, (r, c) => r === c ? 1 : 0);  
console.log("Identity matrix:");  
identityMatrix.forEach(row => console.log(row));
```

```
const multiplicationTable = createMatrix(5, 5, (r, c) => (r + 1) * (c + 1));  
console.log("\n5x5 Multiplication table:");  
multiplicationTable.forEach(row =>  
  console.log(row.map(n => n.toString().padStart(4, ' ')).join(" "))  
);
```

```
console.log("\n--- Practical: Pagination with Array.from() ---");
```

ASSIGNMENT – 1 & 2

```
function paginate(items, pageSize) {  
  const pages = Math.ceil(items.length / pageSize);  
  return Array.from({ length: pages }, (_, i) =>  
    items.slice(i * pageSize, (i + 1) * pageSize)  
  );  
}
```

```
const allItems = Array.from({ length: 25 }, (_, i) => `Item ${i + 1}`);  
const pages = paginate(allItems, 10);  
pages.forEach((page, i) => {  
  console.log(`Page ${i + 1}:`, page);  
});
```

ARRAY-LIKE OBJECTS AND CONVERSION

// Program 50: Working with array-like objects

```
console.log("--- Understanding Array-like Objects ---");
```

// Creating array-like object

```
const arrayLike = {  
  0: 'first',  
  1: 'second',  
  2: 'third',  
  length: 3  
};
```

ASSIGNMENT – 1 & 2

```
console.log("Array-like object:", arrayLike);  
console.log("Is array?", Array.isArray(arrayLike)); // false  
console.log("Has length:", arrayLike.length);  
console.log("Can access by index:", arrayLike[0]);
```

```
console.log("\n--- Common Array-like Objects ---");
```

```
// Arguments object in function  
function demoArguments() {  
    console.log("arguments object:", arguments);  
    console.log("arguments.length:", arguments.length);  
    console.log("arguments[0]:", arguments[0]);  
    console.log("Is array?", Array.isArray(arguments));  
}  
demoArguments('a', 'b', 'c');
```

```
// String as array-like  
const str = "Hello";  
console.log("\nString as array-like:");  
console.log("str.length:", str.length);  
console.log("str[0]:", str[0]);  
console.log("Is array?", Array.isArray(str));
```

```
console.log("\n--- Converting Array-like to Array ---");
```

ASSIGNMENT – 1 & 2

```
const arrayLikeObj = { 0: 'a', 1: 'b', 2: 'c', length: 3 };
```

```
// Method 1: Array.from()
```

```
const asArray1 = Array.from(arrayLikeObj);
```

```
console.log("Array.from():", asArray1);
```

```
// Method 2: Spread operator
```

```
const asArray2 = [...arrayLikeObj]; // Only works with iterables!
```

```
console.log("Can't spread non-iterable array-like");
```

```
// Method 3: Array.prototype.slice.call()
```

```
const asArray3 = Array.prototype.slice.call(arrayLikeObj);
```

```
console.log("slice.call():", asArray3);
```

```
// Method 4: Array.from() with strings
```

```
const strArray = Array.from("Hello");
```

```
console.log("String to array:", strArray);
```

```
console.log("\n--- Borrowing Array Methods ---");
```

```
const list = { 0: 10, 1: 20, 2: 30, length: 3 };
```

```
// forEach
```

```
Array.prototype.forEach.call(list, (item, index) => {
```

```
    console.log(`list[${index}] = ${item}`);
```

ASSIGNMENT – 1 & 2

```
});
```

```
// map
```

```
const doubled = Array.prototype.map.call(list, x => x * 2);  
console.log("Doubled:", doubled);
```

```
// filter
```

```
const filtered = Array.prototype.filter.call(list, x => x > 15);  
console.log("Filtered > 15:", filtered);
```

```
console.log("\n--- HTMLCollection Simulation ---");
```

```
function simulateHTMLCollection() {  
  const divs = {  
    0: { tagName: 'DIV', className: 'container', textContent: 'Div 1' },  
    1: { tagName: 'DIV', className: 'item', textContent: 'Div 2' },  
    2: { tagName: 'DIV', className: 'item', textContent: 'Div 3' },  
    length: 3  
  };  
};
```

```
// Convert and filter
```

```
const divArray = Array.from(divs);  
const items = divArray.filter(div => div.className === 'item');  
console.log("Items with class 'item':", items);
```

```
// Direct iteration using Array methods
```

ASSIGNMENT – 1 & 2

```
Array.prototype.forEach.call(divs, (div, i) => {  
    console.log(` ${i}: ${div.tagName}.${div.className} -  
"${div.textContent}"`);  
});  
}  
simulateHTMLCollection();
```

```
console.log("\n--- Creating Custom Iterable ---");  
const customCollection = {  
    items: ['apple', 'banana', 'orange'],  
    length: 3,  
  
    // Make it iterable  
    [Symbol.iterator]: function() {  
        let index = 0;  
        const items = this.items;  
        return {  
            next: function() {  
                if (index < items.length) {  
                    return { value: items[index++], done: false };  
                }  
                return { done: true };  
            }  
        };  
    }  
}
```


ASSIGNMENT – 1 & 2

```
};
```

```
console.log("Custom iterable can use spread:", [...customCollection]);
```

```
console.log("Can use for...of:");
```

```
for (const item of customCollection) {
```

```
    console.log(` ${item}`);
```

```
}
```

```
console.log("\n--- Practical: Function that accepts array OR array-like ---");
```

```
function processCollection(collection) {
```

```
    // Convert to array if needed
```

```
    const items = Array.isArray(collection) ? collection :  
    Array.from(collection);
```

```
    return {
```

```
        count: items.length,
```

```
        first: items[0],
```

```
        last: items[items.length - 1],
```

```
        all: items
```

```
    };
```

```
}
```

```
console.log("Process array:", processCollection([1, 2, 3]));
```

```
console.log("Process array-like:", processCollection({0: 'x', 1: 'y', length: 2}));
```

```
console.log("Process string:", processCollection("Hi"));
```

ASSIGNMENT – 1 & 2

DOM SELECTION METHODS - GETELEMENTBYID, QUERYSELECTOR

// Program 51: Selecting DOM elements

// Note: This program expects to run in a browser with HTML elements

// Wait for DOM to load

```
document.addEventListener('DOMContentLoaded', function() {
```

```
    console.log("--- getElementById ---");
```

```
    const header = document.getElementById('main-header');
```

```
    if (header) {
```

```
        console.log("Found header:", header);
```

```
        console.log("Header text:", header.textContent);
```

```
    }
```

```
    console.log("\n--- getElementsByClassName ---");
```

```
    const items = document.getElementsByClassName('item');
```

```
    console.log("Items (HTMLCollection):", items);
```

```
    console.log("Number of items:", items.length);
```

```
    // HTMLCollection is live!
```

```
    for (let i = 0; i < items.length; i++) {
```

```
        console.log(`Item ${i}:`, items[i].textContent);
```

```
    }
```

```
    console.log("\n--- getElementsByTagName ---");
```

ASSIGNMENT – 1 & 2

```
const paragraphs = document.getElementsByTagName('p');  
console.log("Paragraphs:", paragraphs);  
console.log("First paragraph:", paragraphs[0]?.textContent);
```

```
console.log("\n--- querySelector ---");  
const firstItem = document.querySelector('.item');  
console.log("First .item:", firstItem?.textContent);
```

```
const nestedElement = document.querySelector('.container .nested');  
console.log("Nested element:", nestedElement?.textContent);
```

DOM CONTENT MANIPULATION - TEXTCONTENT, INNERHTML

// Program 51: Selecting DOM elements

// Note: This program expects to run in a browser with HTML elements

// Wait for DOM to load

```
document.addEventListener('DOMContentLoaded', function() {
```

```
    console.log("--- getElementById ---");  
    const header = document.getElementById('main-header');  
    if (header) {  
        console.log("Found header:", header);  
        console.log("Header text:", header.textContent);  
    }
```

```
    console.log("\n--- getElementsByClassName ---");
```

ASSIGNMENT – 1 & 2

```
const items = document.getElementsByClassName('item');
console.log("Items (HTMLCollection):", items);
console.log("Number of items:", items.length);
```

```
// HTMLCollection is live!
for (let i = 0; i < items.length; i++) {
    console.log(`Item ${i}:`, items[i].textContent);
}
```

```
console.log("\n--- getElementsByTagName ---");
const paragraphs = document.getElementsByTagName('p');
console.log("Paragraphs:", paragraphs);
console.log("First paragraph:", paragraphs[0]?.textContent);
```

```
console.log("\n--- querySelector ---");
const firstItem = document.querySelector('.item');
console.log("First .item:", firstItem?.textContent);
```

```
const nestedElement = document.querySelector('.container .nested');
console.log("Nested element:", nestedElement?.textContent);
```

DOM ATTRIBUTES AND CLASSES - SETATTRIBUTE, CLASSLIST

// Program 53: Working with attributes and CSS classes

```
document.addEventListener('DOMContentLoaded', function() {
```

ASSIGNMENT – 1 & 2

```
console.log("--- Working with Attributes ---");

const image = document.getElementById('demo-image');

if (image) {
    // getAttribute
    console.log("src:", image.getAttribute('src'));
    console.log("alt:", image.getAttribute('alt'));
    console.log("data-id:", image.getAttribute('data-id'));

    // setAttribute
    image.setAttribute('alt', 'Updated description');
    image.setAttribute('data-loaded', 'true');
    image.setAttribute('loading', 'lazy');

    // hasAttribute
    console.log("Has alt?", image.hasAttribute('alt'));
    console.log("Has title?", image.hasAttribute('title'));

    // removeAttribute
    image.removeAttribute('data-id');
    console.log("After remove data-id:", image.hasAttribute('data-id'));

    // Direct property access (for standard attributes)
```

ASSIGNMENT – 1 & 2

```
console.log("Direct src:", image.src);
console.log("Direct alt:", image.alt);
image.title = "Image title set directly";
}

console.log("\n--- Data Attributes ---");
const userCard = document.getElementById('user-card');

if (userCard) {
  // Using dataset
  console.log("Dataset:", userCard.dataset);
  console.log("User ID:", userCard.dataset.userId);
  console.log("User Role:", userCard.dataset.userRole);

  // Setting dataset
  userCard.dataset.lastLogin = new Date().toISOString();
  userCard.dataset.loginCount = '42';
  console.log("Updated dataset:", userCard.dataset);

  // dataset automatically converts kebab-case to camelCase
  console.log("user-role becomes:", userCard.dataset.userRole);
}

console.log("\n--- Working with Classes - className ---");
const box = document.getElementById('demo-box');
```

ASSIGNMENT – 1 & 2

```
if (box) {  
    // className (string manipulation - old way)  
    console.log("Original className:", box.className);  
    box.className += ' highlighted';  
    console.log("After adding class:", box.className);  
    box.className = box.className.replace('highlighted', '').trim();  
    console.log("After removing class:", box.className);  
}
```

```
console.log("\n--- Working with Classes - classList (Modern) ---");  
const modernBox = document.getElementById('modern-box');
```

```
if (modernBox) {  
    // add  
    modernBox.classList.add('active', 'visible');  
    console.log("After add:", modernBox.className);  
  
    // remove  
    modernBox.classList.remove('visible');  
    console.log("After remove:", modernBox.className);  
  
    // toggle  
    modernBox.classList.toggle('selected');  
    console.log("After toggle (add):", modernBox.className);
```

ASSIGNMENT – 1 & 2

```
modernBox.classList.toggle('selected');
console.log("After toggle (remove):", modernBox.className);

// toggle with condition
const isDarkMode = true;
modernBox.classList.toggle('dark-mode', isDarkMode);
console.log("Conditional toggle:", modernBox.className);

// contains
console.log("Contains 'active'", modernBox.classList.contains('active'));
console.log("Contains 'hidden'",
modernBox.classList.contains('hidden'));

// replace
modernBox.classList.replace('active', 'enabled');
console.log("After replace:", modernBox.className);

// length and iteration
console.log("Number of classes:", modernBox.classList.length);
console.log("All classes:", [...modernBox.classList]);
}

console.log("\n--- Style Property ---");
const styledElement = document.getElementById('styled-element');
```


ASSIGNMENT – 1 & 2

```
if (styledElement) {  
    // Individual styles  
    styledElement.style.backgroundColor = '#f0f0f0';  
    styledElement.style.color = '#333';  
    styledElement.style.padding = '15px';  
    styledElement.style.borderRadius = '5px';  
    styledElement.style.border = '1px solid #ccc';  
  
    // Multiple styles with cssText  
    // styledElement.style.cssText = `  
    //   background-color: #e0e0e0;  
    //   color: #000;  
    //   font-size: 18px;  
    // `;  
  
    // Getting computed styles  
    const computed = window.getComputedStyle(styledElement);  
    console.log("Computed background:", computed.backgroundColor);  
    console.log("Computed font-size:", computed.fontSize);  
}  
  
console.log("\n--- Practical: Interactive Element ---");  
const interactiveBtn = document.getElementById('interactive-btn');  
const statusDiv = document.getElementById('status');
```

ASSIGNMENT – 1 & 2

```
if (interactiveBtn && statusDiv) {  
    let clickCount = 0;  
  
    interactiveBtn.addEventListener('click', function() {  
        clickCount++;  
  
        // Update attributes  
        this.setAttribute('data-clicks', clickCount);  
  
        // Update classes  
        this.classList.toggle('clicked');  
  
        if (clickCount >= 5) {  
            this.classList.add('expert');  
        }  
  
        // Update content  
        statusDiv.textContent = `Clicked ${clickCount} time${clickCount  
!= 1 ? 's' : ''}`;  
        statusDiv.classList.add('updated');  
  
        // Update dataset  
        statusDiv.dataset.timestamp = Date.now();  
    });  
}
```

ASSIGNMENT – 1 & 2

```
});
```

// HTML structure for this program:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>DOM Attributes and Classes</title>
```

```
  <style>
```

```
    .box { width: 100px; height: 100px; background: #ddd; margin: 10px;
  }
```

```
    .highlighted { border: 3px solid gold; }
```

```
    .active { box-shadow: 0 0 10px rgba(0,0,0,0.3); }
```

```
    .enabled { opacity: 1; }
```

```
    .dark-mode { background: #333 !important; color: white; }
```

```
    .clicked { background: #4CAF50; color: white; }
```

```
    .expert { background: #2196F3; font-weight: bold; }
```

```
    .updated { animation: pulse 0.5s; }
```

```
    @keyframes pulse { 0%,100% { opacity: 1; } 50% { opacity: 0.5; } }
```

```
  </style>
```

```
</head>
```

```
<body>
```

```
  <h1>Attributes and Classes Demo</h1>
```

ASSIGNMENT – 1 & 2

```

```

```
<div id="user-card" data-user-id="42" data-user-role="admin">  
  User Card  
</div>
```

```
<div id="demo-box" class="box">Old Way</div>  
<div id="modern-box" class="box">Modern Way</div>  
<div id="styled-element">Styled Element</div>
```

```
<button id="interactive-btn" data-clicks="0">Click Me</button>  
<div id="status">Not clicked yet</div>
```

```
<script src="program53.js"></script>  
</body>  
</html>
```

DOM ELEMENT CREATION AND REMOVAL - CREATEELEMENT, APPENDCHILD, REMOVE

// Program 54: Creating and removing DOM elements

```
document.addEventListener('DOMContentLoaded', function() {
```

ASSIGNMENT – 1 & 2

```
console.log("--- Creating Elements ---");

// createElement
const newDiv = document.createElement('div');
console.log("Created element:", newDiv);

// Set properties
newDiv.id = 'dynamic-div';
newDiv.className = 'dynamic-box';
newDiv.textContent = 'I was created dynamically!';
newDiv.setAttribute('data-created', new Date().toISOString());

console.log("Configured element:", newDiv);

console.log("\n--- Appending Elements ---");
const container = document.getElementById('element-container');

if (container) {
    // appendChild (traditional)
    container.appendChild(newDiv);
    console.log("Element appended with appendChild");

    // append (modern - accepts multiple nodes and strings)
    const span1 = document.createElement('span');
    span1.textContent = 'First ';
```

ASSIGNMENT – 1 & 2

```
const span2 = document.createElement('span');
span2.textContent = 'Second ';

container.append(span1, span2, 'Text node!');
console.log("Multiple items appended with append()");
}

console.log("\n--- Insertion Methods ---");
const target = document.getElementById('insertion-target');

if (target) {
    // before - inserts before the element
    const beforeElement = document.createElement('div');
    beforeElement.textContent = 'Before element';
    beforeElement.className = 'inserted-before';
    target.before(beforeElement);

    // after - inserts after the element
    const afterElement = document.createElement('div');
    afterElement.textContent = 'After element';
    afterElement.className = 'inserted-after';
    target.after(afterElement);

    // prepend - inserts as first child
    const prependElement = document.createElement('div');
```

ASSIGNMENT – 1 & 2

```
prependElement.textContent = 'First child (prepended)';  
prependElement.className = 'prepended';  
target.prepend(prependElement);
```

```
// append - inserts as last child  
const appendElement = document.createElement('div');  
appendElement.textContent = 'Last child (appended)';  
appendElement.className = 'appended';  
target.append(appendElement);  
}
```

```
console.log("\n--- insertBefore (traditional) ---");  
const parent = document.getElementById('parent-list');
```

```
if (parent) {  
    const newItem = document.createElement('li');  
    newItem.textContent = 'Inserted Item';  
    newItem.className = 'inserted';  
  
    const referenceNode = parent.children[1]; // Second child  
    parent.insertBefore(newItem, referenceNode);  
    console.log("Item inserted before second child");  
}
```

```
console.log("\n--- Cloning Elements ---");
```

ASSIGNMENT – 1 & 2

```
const original = document.getElementById('clone-source');
```

```
if (original) {
```

```
    // Shallow clone (element only, no children)
```

```
    const shallowClone = original.cloneNode(false);
```

```
    shallowClone.id = 'shallow-clone';
```

```
    shallowClone.textContent = 'Shallow Clone';
```

```
    original.after(shallowClone);
```

```
    // Deep clone (element and all children)
```

```
    const deepClone = original.cloneNode(true);
```

```
    deepClone.id = 'deep-clone';
```

```
    original.after(deepClone);
```

```
    console.log("Clones created");
```

```
}
```

```
console.log("\n--- Removing Elements ---");
```

```
// remove() - modern method
```

```
const removeMe = document.getElementById('remove-me');
```

```
if (removeMe) {
```

```
    removeMe.remove();
```

```
    console.log("Element removed with remove()");
```

```
}
```


ASSIGNMENT – 1 & 2

```
// removeChild - traditional method

const parentForRemoval = document.getElementById('parent-for-removal');

const childToRemove = document.getElementById('child-to-remove');

if (parentForRemoval && childToRemove) {
    parentForRemoval.removeChild(childToRemove);
    console.log("Child removed with removeChild()");
}

console.log("\n--- Replacing Elements ---");
const oldElement = document.getElementById('old-element');

if (oldElement) {
    const newElement = document.createElement('div');
    newElement.id = 'new-element';
    newElement.textContent = 'I replaced the old element!';
    newElement.className = 'replacement';

    // replaceWith - modern method
    oldElement.replaceWith(newElement);
    console.log("Element replaced with replaceWith()");
}
```

ASSIGNMENT – 1 & 2

```
console.log("\n--- Practical: Dynamic List Builder ---");
const listBuilder = document.getElementById('list-builder');
const addBtn = document.getElementById('add-item-btn');
const clearBtn = document.getElementById('clear-items-btn');
const itemInput = document.getElementById('item-input');
const dynamicList = document.getElementById('dynamic-list');
const itemCountSpan = document.getElementById('item-count');

if (addBtn && dynamicList) {
  function updateItemCount() {
    if (itemCountSpan) {
      itemCountSpan.textContent = dynamicList.children.length;
    }
  }

  function createListItem(text) {
    const li = document.createElement('li');
    li.className = 'dynamic-item';

    const textSpan = document.createElement('span');
    textSpan.textContent = text;
    textSpan.className = 'item-text';

    const deleteBtn = document.createElement('button');
    deleteBtn.textContent = '×';
```

ASSIGNMENT – 1 & 2

```
deleteBtn.className = 'delete-btn';

deleteBtn.setAttribute('aria-label', 'Delete item');


deleteBtn.addEventListener('click', function() {
    li.remove();
    updateItemCount();
});


const editBtn = document.createElement('button');
editBtn.textContent = '✎';
editBtn.className = 'edit-btn';


editBtn.addEventListener('click', function() {
    const newText = prompt('Edit item:', textSpan.textContent);
    if (newText && newText.trim()) {
        textSpan.textContent = newText.trim();
    }
});


li.append(textSpan, editBtn, deleteBtn);


// Double-click to toggle completed
li.addEventListener('dblclick', function() {
    this.classList.toggle('completed');
});
```

ASSIGNMENT – 1 & 2

```
    return li;
}

addBtn.addEventListener('click', function() {
    const text = itemInput ? itemInput.value.trim() : "";
    if (text) {
        const newItem = createListItem(text);
        dynamicList.appendChild(newItem);
        updateItemCount();
        if (itemInput) itemInput.value = "";
    }
});
```

```
if (clearBtn) {
    clearBtn.addEventListener('click', function() {
        dynamicList.innerHTML = "";
        updateItemCount();
    });
}
```

```
// Add some sample items
const sampleItems = ['Learn JavaScript', 'Build a project', 'Master
DOM'];
sampleItems.forEach(text => {
```

ASSIGNMENT – 1 & 2

```
        dynamicList.appendChild(createListItem(text));
    });
    updateItemCount();
}
```

```
console.log("\n--- DocumentFragment for Performance ---");
const fragmentContainer = document.getElementById('fragment-demo');
```

```
if (fragmentContainer) {
    console.time('Without Fragment');
    for (let i = 0; i < 1000; i++) {
        const div = document.createElement('div');
        div.textContent = `Item ${i}`;
        // DON'T DO THIS - causes reflow each time
        // fragmentContainer.appendChild(div);
    }
    console.timeEnd('Without Fragment');
```

```
    console.time('With Fragment');
    const fragment = document.createDocumentFragment();
    for (let i = 0; i < 1000; i++) {
        const div = document.createElement('div');
        div.textContent = `Item ${i}`;
        fragment.appendChild(div);
    }
```

ASSIGNMENT – 1 & 2

```
fragmentContainer.appendChild(fragment); // Single reflow!  
console.timeEnd('With Fragment');  
}  
});
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>DOM Creation and Removal</title>
```

```
<style>
```

```
.dynamic-box { background: #e3f2fd; padding: 10px; margin: 10px; }
```

```
.inserted-before { background: #fff3e0; padding: 5px; }
```

```
.inserted-after { background: #e8f5e9; padding: 5px; }
```

```
.prepended { background: #fce4ec; padding: 5px; }
```

```
.appended { background: #f3e5f5; padding: 5px; }
```

```
.replacement { background: #ffeb3b; padding: 10px; }
```

```
.dynamic-item {
```

```
  display: flex;
```

```
  align-items: center;
```

```
  gap: 10px;
```

```
  padding: 8px;
```

```
  margin: 5px 0;
```

```
  background: #f5f5f5;
```

```
  border-radius: 4px;
```

ASSIGNMENT – 1 & 2

```
}  
.dynamic-item.completed {  
    background: #c8e6c9;  
    text-decoration: line-through;  
}  
.item-text { flex: 1; }  
.delete-btn, .edit-btn {  
    padding: 4px 8px;  
    cursor: pointer;  
    border: none;  
    border-radius: 3px;  
}  
.delete-btn { background: #ffcdd2; }  
.edit-btn { background: #bbdefb; }  
.delete-btn:hover { background: #ef9a9a; }  
.edit-btn:hover { background: #90caf9; }  
</style>  
</head>  
<body>  
    <h1>Element Creation and Removal</h1>  
  
    <div id="element-container" style="border: 1px solid #ccc; padding:  
10px; margin: 10px;">  
        <h3>Container</h3>  
    </div>
```


ASSIGNMENT – 1 & 2

</div>

<div id="old-element">I am old and will be replaced</div>

<hr>

<h2>Dynamic List Builder</h2>

<div id="list-builder">

 <input type="text" id="item-input" placeholder="Enter item...">

 <button id="add-item-btn">Add Item</button>

 <button id="clear-items-btn">Clear All</button>

 <p>Total items: 0 </p>

 <ul id="dynamic-list">

</div>

<hr>

<h2>Performance Test</h2>

<div id="fragment-demo"></div>

 <script src="program54.js"></script>

</body>

</html>

[DOM EVENTS - ADDEVENTLISTENER AND EVENT OBJECT](#)

ASSIGNMENT – 1 & 2

// Program 56: Event propagation and delegation

```
document.addEventListener('DOMContentLoaded', function() {

    console.log("--- Event Bubbling ---");

    const grandparent = document.getElementById('grandparent');
    const parent = document.getElementById('parent');
    const child = document.getElementById('child');

    if (grandparent && parent && child) {
        grandparent.addEventListener('click', function(e) {
            console.log('Grandparent clicked (bubbling phase)');
            console.log('Target:', e.target.id);
            console.log('CurrentTarget:', e.currentTarget.id);
        });

        parent.addEventListener('click', function(e) {
            console.log('Parent clicked (bubbling phase)');
        });

        child.addEventListener('click', function(e) {
            console.log('Child clicked (bubbling phase)');
        });
    }
}
```

ASSIGNMENT – 1 & 2

```
console.log("\n--- Event Capturing ---");
```

```
if (grandparent) {  
  grandparent.addEventListener('click', function(e) {  
    console.log('Grandparent clicked (CAPTURING phase)');  
  }, { capture: true });
```

```
  
  parent.addEventListener('click', function(e) {  
    console.log('Parent clicked (CAPTURING phase)');  
  }, true); // Alternative syntax  
}
```

```
console.log("\n--- Stopping Propagation ---");
```

```
const stopBtn = document.getElementById('stop-propagation');  
const stopContainer = document.getElementById('stop-container');
```

```
if (stopContainer) {  
  stopContainer.addEventListener('click', function() {  
    console.log('Container clicked - this should NOT fire when button  
clicked');  
  });  
}
```

ASSIGNMENT – 1 & 2

```
if (stopBtn) {  
  stopBtn.addEventListener('click', function(e) {  
    console.log('Button clicked');  
    e.stopPropagation(); // Stops bubbling  
    console.log('Propagation stopped!');  
  });  
}
```

```
console.log("\n--- Event Delegation Pattern ---");
```

```
const todoList = document.getElementById('todo-list');  
const addTodoBtn = document.getElementById('add-todo');  
const todoInput = document.getElementById('todo-input');  
const todoCount = document.getElementById('todo-count');
```

```
// Single event listener on parent (delegation)
```

```
if (todoList) {  
  todoList.addEventListener('click', function(e) {  
    const target = e.target;  
  
    // Check if delete button was clicked  
    if (target.classList.contains('delete-btn')) {  
      const todoItem = target.closest('.todo-item');  
      if (todoItem) {  
        todoItem.remove();  
      }  
    }  
  });  
}
```

ASSIGNMENT – 1 & 2

```
        updateTodoCount();
        console.log('Todo deleted via delegation');
    }
}

// Check if checkbox was clicked
if (target.classList.contains('todo-checkbox')) {
    const todoItem = target.closest('.todo-item');
    if (todoItem) {
        todoItem.classList.toggle('completed', target.checked);
        updateTodoCount();
        console.log('Todo status updated via delegation');
    }
}

// Check if edit button was clicked
if (target.classList.contains('edit-btn')) {
    const todoItem = target.closest('.todo-item');
    const todoText = todoItem.querySelector('.todo-text');
    const newText = prompt('Edit todo:', todoText.textContent);
    if (newText && newText.trim()) {
        todoText.textContent = newText.trim();
    }
}

});
```

ASSIGNMENT – 1 & 2

```
}
```

```
function updateTodoCount() {  
  if (todoCount && todoList) {  
    const total = todoList.children.length;  
    const completed = todoList.querySelectorAll('.todo-item.completed').length;  
    todoCount.textContent = ` ${completed}/${total} completed `;  
  }  
}
```

```
function createTodoItem(text) {  
  const li = document.createElement('li');  
  li.className = 'todo-item';  
  li.innerHTML = `  
    <input type="checkbox" class="todo-checkbox">  
    <span class="todo-text"> ${text}</span>  
    <button class="edit-btn">Edit</button>  
    <button class="delete-btn">Delete</button>  
  `;  
  return li;  
}
```

```
if (addTodoBtn && todoInput && todoList) {  
  addTodoBtn.addEventListener('click', function() {
```

ASSIGNMENT – 1 & 2

```
const text = todoInput.value.trim();

if (text) {
  todoList.appendChild(createTodoItem(text));
  todoInput.value = "";
  updateTodoCount();
}

});

// Add sample todos
['Learn Event Delegation', 'Master DOM Manipulation', 'Build a Todo
App'].forEach(text => {
  todoList.appendChild(createTodoItem(text));
});
updateTodoCount();
}

console.log("\n--- Dynamic Elements and Delegation ---");

const dynamicContainer = document.getElementById('dynamic-
container');

const addDynamicBtn = document.getElementById('add-dynamic-btn');

if (dynamicContainer) {
  // Delegated listener - works for future elements!
  dynamicContainer.addEventListener('click', function(e) {
```

ASSIGNMENT – 1 & 2

```
if (e.target.classList.contains('dynamic-btn')) {  
    console.log('Dynamic button clicked:', e.target.dataset.id);  
    alert(`Button ${e.target.dataset.id} clicked!`);  
}  
});  
  
if (addDynamicBtn) {  
    let counter = 0;  
    addDynamicBtn.addEventListener('click', function() {  
        counter++;  
        const btn = document.createElement('button');  
        btn.className = 'dynamic-btn';  
        btn.dataset.id = counter;  
        btn.textContent = `Dynamic Button ${counter}`;  
        dynamicContainer.appendChild(btn);  
        console.log(`Added button ${counter} - delegation will work!`);  
    });  
  
    // Add initial button  
    addDynamicBtn.click();  
}  
}
```

```
console.log("\n--- Event Delegation with Data Attributes ---");
```


ASSIGNMENT – 1 & 2

```
const productGrid = document.getElementById('product-grid');
```

```
const cartDisplay = document.getElementById('cart-display');
```

```
let cart = [];
```

```
const products = [
```

```
  { id: 1, name: 'Laptop', price: 999 },
```

```
  { id: 2, name: 'Mouse', price: 25 },
```

```
  { id: 3, name: 'Keyboard', price: 75 },
```

```
  { id: 4, name: 'Monitor', price: 299 }
```

```
];
```

```
if (productGrid) {
```

```
  // Render products
```

```
  products.forEach(product => {
```

```
    const card = document.createElement('div');
```

```
    card.className = 'product-card';
```

```
    card.dataset.productId = product.id;
```

```
    card.dataset.price = product.price;
```

```
    card.innerHTML = `
```

```
      <h3>${product.name}</h3>
```

```
      <p>${product.price}</p>
```

```
      <button class="add-to-cart-btn" data-action="add">Add to  
Cart</button>
```

```
      <button class="view-details-btn" data-  
action="view">Details</button>
```

ASSIGNMENT – 1 & 2

```
`;  
productGrid.appendChild(card);  
});  
  
// Single delegated listener for all product actions  
productGrid.addEventListener('click', function(e) {  
    const productCard = e.target.closest('.product-card');  
    if (!productCard) return;  
  
    const productId = parseInt(productCard.dataset.productId);  
    const product = products.find(p => p.id === productId);  
    const action = e.target.dataset.action;  
  
    switch(action) {  
        case 'add':  
            cart.push(product);  
            updateCartDisplay();  
            console.log(`Added ${product.name} to cart`);  
            break;  
        case 'view':  
            alert(`${product.name}\nPrice: ${product.price}`);  
            break;  
    }  
});  
}
```

ASSIGNMENT – 1 & 2

```
function updateCartDisplay() {  
  if (cartDisplay) {  
    const total = cart.reduce((sum, item) => sum + item.price, 0);  
    cartDisplay.innerHTML = `  
      <h3>Cart (${cart.length} items)</h3>  
      <p>Total: ${total}</p>  
      <ul>  
        ${cart.map(item => `<li>${item.name} -  
${item.price}</li>`).join("")}  
      </ul>  
    `;  
  }  
}
```

```
console.log("\n--- Preventing Default Behavior ---");
```

```
const linkDemo = document.getElementById('link-demo');
```

```
const formDemo = document.getElementById('prevent-form-demo');
```

```
if (linkDemo) {  
  linkDemo.addEventListener('click', function(e) {  
    e.preventDefault();  
    console.log('Link navigation prevented');  
    alert('Navigation prevented! Check console.');
```

ASSIGNMENT – 1 & 2

```
});  
}  
  
if (formDemo) {  
  formDemo.addEventListener('submit', function(e) {  
    e.preventDefault();  
    console.log('Form submission prevented');  
  
    // Custom validation  
    const email = this.querySelector('[name="email"]');  
    if (email && !email.value.includes('@')) {  
      alert('Please enter a valid email!');  
      return;  
    }  
  
    alert('Form validated! (Still prevented from actual submit)');  
  });  
}  
});
```

// HTML structure for this program:

```
/*  
<!DOCTYPE html>  
<html>  
<head>
```

ASSIGNMENT – 1 & 2

<title>Event Delegation and Bubbling</title>

<style>

#grandparent { padding: 30px; background: #ffcdd2; }

#parent { padding: 30px; background: #c8e6c9; }

#child { padding: 30px; background: #bbdefb; cursor: pointer; }

#stop-container { padding: 20px; background: #fff3e0; }

#stop-propagation { margin: 10px; }

.todo-item {

display: flex;

align-items: center;

gap: 10px;

padding: 10px;

margin: 5px 0;

background: #f5f5f5;

border-radius: 4px;

}

.todo-item.completed .todo-text {

text-decoration: line-through;

color: #999;

}

.todo-text { flex: 1; }

.dynamic-btn { margin: 5px; padding: 8px 16px; }

ASSIGNMENT – 1 & 2

```
.product-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));
  gap: 15px;
}

.product-card {
  border: 1px solid #ddd;
  padding: 15px;
  border-radius: 8px;
  background: white;
}

.product-card button { margin: 5px; }
</style>
</head>
<body>
  <h1>Event Delegation and Bubbling</h1>

  <h2>Event Bubbling Demo</h2>
  <div id="grandparent">
    Grandparent
    <div id="parent">
      Parent
      <div id="child">Child (Click me)</div>
    </div>
```

ASSIGNMENT – 1 & 2

</div>

<h2>Stop Propagation</h2>

<div id="stop-container">

Container (click me)

<button id="stop-propagation">Stop Propagation Button</button>

</div>

<h2>Event Delegation - Todo List</h2>

<div>

<input type="text" id="todo-input" placeholder="Enter todo...">

<button id="add-todo">Add Todo</button>

</div>

<ul id="todo-list">

<h2>Dynamic Elements</h2>

<div id="dynamic-container"></div>

<button id="add-dynamic-btn">Add Dynamic Button</button>

<h2>Product Grid (Delegation)</h2>

<div id="product-grid" class="product-grid"></div>

<div id="cart-display"></div>

<h2>Prevent Default</h2>

ASSIGNMENT – 1 & 2

```
<a href="https://example.com" id="link-demo">This link won't  
navigate</a>
```

```
<form id="prevent-form-demo">  
  <input type="email" name="email" placeholder="Enter email">  
  <button type="submit">Submit (Prevented)</button>  
</form>
```

```
<script src="program56.js"></script>  
</body>  
</html>  
*/
```

EVENT DELEGATION AND BUBBLING

// Program 56: Event propagation and delegation

```
document.addEventListener('DOMContentLoaded', function() {  
  
  console.log("--- Event Bubbling ---");  
  
  const grandparent = document.getElementById('grandparent');  
  const parent = document.getElementById('parent');  
  const child = document.getElementById('child');  
  
  if (grandparent && parent && child) {  
    grandparent.addEventListener('click', function(e) {  
      console.log('Grandparent clicked (bubbling phase)');    });  
  }  
});
```


ASSIGNMENT – 1 & 2

```
    console.log('Target:', e.target.id);
    console.log('CurrentTarget:', e.currentTarget.id);
  });

  parent.addEventListener('click', function(e) {
    console.log('Parent clicked (bubbling phase)');
  });

  child.addEventListener('click', function(e) {
    console.log('Child clicked (bubbling phase)');
  });
}

console.log("\n--- Event Capturing ---");

if (grandparent) {
  grandparent.addEventListener('click', function(e) {
    console.log('Grandparent clicked (CAPTURING phase)');
  }, { capture: true });

  parent.addEventListener('click', function(e) {
    console.log('Parent clicked (CAPTURING phase)');
  }, true); // Alternative syntax
}
```

ASSIGNMENT – 1 & 2

```
console.log("\n--- Stopping Propagation ---");
```

```
const stopBtn = document.getElementById('stop-propagation');
```

```
const stopContainer = document.getElementById('stop-container');
```

```
if (stopContainer) {  
    stopContainer.addEventListener('click', function() {  
        console.log('Container clicked - this should NOT fire when button  
clicked');  
    });  
}
```

```
if (stopBtn) {  
    stopBtn.addEventListener('click', function(e) {  
        console.log('Button clicked');  
        e.stopPropagation(); // Stops bubbling  
        console.log('Propagation stopped!');  
    });  
}
```

```
console.log("\n--- Event Delegation Pattern ---");
```

```
const todoList = document.getElementById('todo-list');
```

```
const addTodoBtn = document.getElementById('add-todo');
```

```
const todoInput = document.getElementById('todo-input');
```

ASSIGNMENT – 1 & 2

```
const todoCount = document.getElementById('todo-count');
```

```
// Single event listener on parent (delegation)
```

```
if (todoList) {
```

```
  todoList.addEventListener('click', function(e) {
```

```
    const target = e.target;
```

```
    // Check if delete button was clicked
```

```
    if (target.classList.contains('delete-btn')) {
```

```
      const todoItem = target.closest('.todo-item');
```

```
      if (todoItem) {
```

```
        todoItem.remove();
```

```
        updateTodoCount();
```

```
        console.log('Todo deleted via delegation');
```

```
      }
```

```
    }
```

```
    // Check if checkbox was clicked
```

```
    if (target.classList.contains('todo-checkbox')) {
```

```
      const todoItem = target.closest('.todo-item');
```

```
      if (todoItem) {
```

```
        todoItem.classList.toggle('completed', target.checked);
```

```
        updateTodoCount();
```

```
        console.log('Todo status updated via delegation');
```

```
      }
```

ASSIGNMENT – 1 & 2

```
}
```

```
// Check if edit button was clicked
```

```
if (target.classList.contains('edit-btn')) {
```

```
    const todoItem = target.closest('.todo-item');
```

```
    const todoText = todoItem.querySelector('.todo-text');
```

```
    const newText = prompt('Edit todo:', todoText.textContent);
```

```
    if (newText && newText.trim()) {
```

```
        todoText.textContent = newText.trim();
```

```
    }
```

```
}
```

```
});
```

```
}
```

```
function updateTodoCount() {
```

```
    if (todoCount && todoList) {
```

```
        const total = todoList.children.length;
```

```
        const completed = todoList.querySelectorAll('.todo-item.completed').length;
```

```
        todoCount.textContent = ` ${completed}/${total} completed `;
```

```
    }
```

```
}
```

```
function createTodoItem(text) {
```

```
    const li = document.createElement('li');
```

ASSIGNMENT – 1 & 2

```
li.className = 'todo-item';

li.innerHTML = `
  <input type="checkbox" class="todo-checkbox">
  <span class="todo-text">${text}</span>
  <button class="edit-btn">Edit</button>
  <button class="delete-btn">Delete</button>
`;

return li;
}

if (addTodoBtn && todoInput && todoList) {
  addTodoBtn.addEventListener('click', function() {
    const text = todoInput.value.trim();
    if (text) {
      todoList.appendChild(createTodoItem(text));
      todoInput.value = "";
      updateTodoCount();
    }
  });

  // Add sample todos
  ['Learn Event Delegation', 'Master DOM Manipulation', 'Build a Todo App'].forEach(text => {
    todoList.appendChild(createTodoItem(text));
  });
}
```

ASSIGNMENT – 1 & 2

```
    updateTodoCount();  
}
```

```
console.log("\n--- Dynamic Elements and Delegation ---");
```

```
const dynamicContainer = document.getElementById('dynamic-  
container');  
const addDynamicBtn = document.getElementById('add-dynamic-btn');
```

```
if (dynamicContainer) {  
    // Delegated listener - works for future elements!  
    dynamicContainer.addEventListener('click', function(e) {  
        if (e.target.classList.contains('dynamic-btn')) {  
            console.log('Dynamic button clicked:', e.target.dataset.id);  
            alert(`Button ${e.target.dataset.id} clicked!`);  
        }  
    });
```

```
if (addDynamicBtn) {  
    let counter = 0;  
    addDynamicBtn.addEventListener('click', function() {  
        counter++;  
        const btn = document.createElement('button');  
        btn.className = 'dynamic-btn';  
        btn.dataset.id = counter;
```

ASSIGNMENT – 1 & 2

```
    btn.textContent = `Dynamic Button ${counter}`;  
    dynamicContainer.appendChild(btn);  
    console.log(`Added button ${counter} - delegation will work!`);  
  });  
  
  // Add initial button  
  addDynamicBtn.click();  
}  
}
```

```
console.log("\n--- Event Delegation with Data Attributes ---");
```

```
const productGrid = document.getElementById('product-grid');  
const cartDisplay = document.getElementById('cart-display');  
let cart = [];
```

```
const products = [  
  { id: 1, name: 'Laptop', price: 999 },  
  { id: 2, name: 'Mouse', price: 25 },  
  { id: 3, name: 'Keyboard', price: 75 },  
  { id: 4, name: 'Monitor', price: 299 }  
];
```

```
if (productGrid) {  
  // Render products
```

ASSIGNMENT – 1 & 2

```
products.forEach(product => {
  const card = document.createElement('div');
  card.className = 'product-card';
  card.dataset.productId = product.id;
  card.dataset.price = product.price;
  card.innerHTML = `
    <h3>${product.name}</h3>
    <p>${product.price}</p>
    <button class="add-to-cart-btn" data-action="add">Add to
Cart</button>
    <button class="view-details-btn" data-
action="view">Details</button>
  `;
  productGrid.appendChild(card);
});

// Single delegated listener for all product actions
productGrid.addEventListener('click', function(e) {
  const productCard = e.target.closest('.product-card');
  if (!productCard) return;

  const productId = parseInt(productCard.dataset.productId);
  const product = products.find(p => p.id === productId);
  const action = e.target.dataset.action;
```


ASSIGNMENT – 1 & 2

```
switch(action) {  
  case 'add':  
    cart.push(product);  
    updateCartDisplay();  
    console.log(`Added ${product.name} to cart`);  
    break;  
  case 'view':  
    alert(`${product.name}\nPrice: ${product.price}`);  
    break;  
}  
});  
}
```

```
function updateCartDisplay() {  
  if (cartDisplay) {  
    const total = cart.reduce((sum, item) => sum + item.price, 0);  
    cartDisplay.innerHTML = `  
      <h3>Cart (${cart.length} items)</h3>  
      <p>Total: ${total}</p>  
      <ul>  
        ${cart.map(item => `<li>${item.name} -  
        ${item.price}</li>`).join("")}  
      </ul>  
    `;  
  }  
}
```

ASSIGNMENT – 1 & 2

```
}
```

```
console.log("\n--- Preventing Default Behavior ---");
```

```
const linkDemo = document.getElementById('link-demo');
```

```
const formDemo = document.getElementById('prevent-form-demo');
```

```
if (linkDemo) {
```

```
    linkDemo.addEventListener('click', function(e) {
```

```
        e.preventDefault();
```

```
        console.log('Link navigation prevented');
```

```
        alert('Navigation prevented! Check console.');
```

```
    });
```

```
}
```

```
if (formDemo) {
```

```
    formDemo.addEventListener('submit', function(e) {
```

```
        e.preventDefault();
```

```
        console.log('Form submission prevented');
```

```
        // Custom validation
```

```
        const email = this.querySelector('[name="email"]');
```

```
        if (email && !email.value.includes('@')) {
```

```
            alert('Please enter a valid email!');
```

```
            return;
```

ASSIGNMENT – 1 & 2

```
}

    alert('Form validated! (Still prevented from actual submit)');
});
}
});

// HTML structure for this program:
/*
<!DOCTYPE html>
<html>
<head>
    <title>Event Delegation and Bubbling</title>
    <style>
        #grandparent { padding: 30px; background: #ffcdd2; }
        #parent { padding: 30px; background: #c8e6c9; }
        #child { padding: 30px; background: #bbdefb; cursor: pointer; }

        #stop-container { padding: 20px; background: #fff3e0; }
        #stop-propagation { margin: 10px; }

        .todo-item {
            display: flex;
            align-items: center;
            gap: 10px;
```

ASSIGNMENT – 1 & 2

```
padding: 10px;
margin: 5px 0;
background: #f5f5f5;
border-radius: 4px;
}

.todo-item.completed .todo-text {
  text-decoration: line-through;
  color: #999;
}

.todo-text { flex: 1; }

.dynamic-btn { margin: 5px; padding: 8px 16px; }

.product-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));
  gap: 15px;
}

.product-card {
  border: 1px solid #ddd;
  padding: 15px;
  border-radius: 8px;
  background: white;
}

.product-card button { margin: 5px; }
```

ASSIGNMENT – 1 & 2

```
</style>
```

```
</head>
```

```
<body>
```

```
<h1>Event Delegation and Bubbling</h1>
```

```
<h2>Event Bubbling Demo</h2>
```

```
<div id="grandparent">
```

```
  Grandparent
```

```
    <div id="parent">
```

```
      Parent
```

```
        <div id="child">Child (Click me)</div>
```

```
      </div>
```

```
    </div>
```

```
<h2>Stop Propagation</h2>
```

```
<div id="stop-container">
```

```
  Container (click me)
```

```
    <button id="stop-propagation">Stop Propagation Button</button>
```

```
</div>
```

```
<h2>Event Delegation - Todo List</h2>
```

```
<div>
```

```
  <input type="text" id="todo-input" placeholder="Enter todo...">
```

```
  <button id="add-todo">Add Todo</button>
```

```
  <span id="todo-count"></span>
```

ASSIGNMENT – 1 & 2

</div>

<ul id="todo-list">

<h2>Dynamic Elements</h2>

<div id="dynamic-container"></div>

<button id="add-dynamic-btn">Add Dynamic Button</button>

<h2>Product Grid (Delegation)</h2>

<div id="product-grid" class="product-grid"></div>

<div id="cart-display"></div>

<h2>Prevent Default</h2>

This link won't
navigate

<form id="prevent-form-demo">

 <input type="email" name="email" placeholder="Enter email">

 <button type="submit">Submit (Prevented)</button>

</form>

<script src="program56.js"></script>

</body>

</html>

*/

[POPULATING A TABLE WITH JAVASCRIPT \(COMPREHENSIVE DOM
EXAMPLE\)](#)

ASSIGNMENT – 1 & 2

// Program 57: Complete table population and management with JavaScript

```
document.addEventListener('DOMContentLoaded', function() {  
  
    // Sample data  
    const students = [  
        { id: 1, firstName: 'John', lastName: 'Doe', age: 20, grade: 'A', email: 'john@example.com', major: 'Computer Science' },  
        { id: 2, firstName: 'Jane', lastName: 'Smith', age: 22, grade: 'A-', email: 'jane@example.com', major: 'Mathematics' },  
        { id: 3, firstName: 'Bob', lastName: 'Johnson', age: 19, grade: 'B+', email: 'bob@example.com', major: 'Physics' },  
        { id: 4, firstName: 'Alice', lastName: 'Williams', age: 21, grade: 'A', email: 'alice@example.com', major: 'Engineering' },  
        { id: 5, firstName: 'Charlie', lastName: 'Brown', age: 20, grade: 'B', email: 'charlie@example.com', major: 'Computer Science' },  
        { id: 6, firstName: 'Diana', lastName: 'Miller', age: 23, grade: 'A-', email: 'diana@example.com', major: 'Biology' },  
        { id: 7, firstName: 'Edward', lastName: 'Davis', age: 22, grade: 'C+', email: 'edward@example.com', major: 'Chemistry' },  
        { id: 8, firstName: 'Fiona', lastName: 'Garcia', age: 20, grade: 'B+', email: 'fiona@example.com', major: 'Mathematics' }  
    ];  
  
    // DOM elements  
    const tableBody = document.getElementById('table-body');  
    const searchInput = document.getElementById('search-input');
```

ASSIGNMENT – 1 & 2

```
const gradeFilter = document.getElementById('grade-filter');
const majorFilter = document.getElementById('major-filter');
const sortSelect = document.getElementById('sort-select');
const addStudentBtn = document.getElementById('add-student-btn');
const studentForm = document.getElementById('student-form');
const formTitle = document.getElementById('form-title');
const cancelFormBtn = document.getElementById('cancel-form');
const statsDisplay = document.getElementById('stats-display');
const exportBtn = document.getElementById('export-btn');
const paginationContainer = document.getElementById('pagination');
```

```
// State
```

```
let currentData = [...students];
let filteredData = [...students];
let editingId = null;
let currentPage = 1;
const rowsPerPage = 5;
```

```
// Initialize filters
```

```
function initializeFilters() {
  const grades = [...new Set(students.map(s => s.grade))].sort();
  const majors = [...new Set(students.map(s => s.major))].sort();

  grades.forEach(grade => {
    const option = document.createElement('option');
```


ASSIGNMENT – 1 & 2

```
option.value = grade;
option.textContent = grade;
if (gradeFilter) gradeFilter.appendChild(option);
});
```

```
majors.forEach(major => {
  const option = document.createElement('option');
  option.value = major;
  option.textContent = major;
  if (majorFilter) majorFilter.appendChild(option);
});
}
```

// Render table with current filtered data

```
function renderTable() {
  if (!tableBody) return;

  const start = (currentPage - 1) * rowsPerPage;
  const end = start + rowsPerPage;
  const pageData = filteredData.slice(start, end);

  if (pageData.length === 0 && currentPage > 1) {
    currentPage--;
    renderTable();
    return;
  }
}
```

ASSIGNMENT – 1 & 2

```
}
```

```
// Use DocumentFragment for performance
```

```
const fragment = document.createDocumentFragment();
```

```
pageData.forEach(student => {
```

```
    const row = document.createElement('tr');
```

```
    row.dataset.id = student.id;
```

```
    // Apply row color based on grade
```

```
    if (student.grade.startsWith('A')) {
```

```
        row.style.backgroundColor = '#e8f5e9';
```

```
    } else if (student.grade.startsWith('B')) {
```

```
        row.style.backgroundColor = '#fff8e1';
```

```
    } else if (student.grade.startsWith('C')) {
```

```
        row.style.backgroundColor = '#ffebee';
```

```
    }
```

```
    row.innerHTML = `
```

```
        <td>${student.id}</td>
```

```
        <td>${student.firstName}</td>
```

```
        <td>${student.lastName}</td>
```

```
        <td>${student.age}</td>
```

ASSIGNMENT – 1 & 2

```
<td><span class="grade-badge grade-  
${student.grade.charAt(0).toLowerCase()}> ${student.grade}</span></td>  
>
```

```
<td>${student.email}</td>
```

```
<td>${student.major}</td>
```

```
<td class="action-cell">
```

```
<button class="edit-btn" data-id="${student.id}"  
title="Edit">✎</button>
```

```
<button class="delete-btn" data-id="${student.id}"  
title="Delete">✕</button>
```

```
</td>
```

```
`;  
`;
```

```
fragment.appendChild(row);  
});
```

```
// Clear and repaint  
tableBody.innerHTML = "";  
tableBody.appendChild(fragment);
```

```
// Update stats and pagination  
updateStats();  
renderPagination();  
}
```

```
// Update statistics
```

ASSIGNMENT – 1 & 2

```
function updateStats() {  
  if (!statsDisplay) return;  
  
  const total = filteredData.length;  
  const avgAge = total > 0 ? (filteredData.reduce((sum, s) => sum +  
s.age, 0) / total).toFixed(1) : 0;  
  const aGrades = filteredData.filter(s => s.grade.startsWith('A')).length;  
  const csCount = filteredData.filter(s => s.major === 'Computer  
Science').length;  
  
  statsDisplay.innerHTML = `  
    <div class="stat-card">  
      <span class="stat-value">${total}</span>  
      <span class="stat-label">Total Students</span>  
    </div>  
    <div class="stat-card">  
      <span class="stat-value">${avgAge}</span>  
      <span class="stat-label">Average Age</span>  
    </div>  
    <div class="stat-card">  
      <span class="stat-value">${aGrades}</span>  
      <span class="stat-label">A Grades</span>  
    </div>  
    <div class="stat-card">  
      <span class="stat-value">${csCount}</span>
```

ASSIGNMENT – 1 & 2

```
<span class="stat-label">CS Majors</span>
</div>
`;
}

// Render pagination controls
function renderPagination() {
  if (!paginationContainer) return;

  const totalPages = Math.ceil(filteredData.length / rowsPerPage);
  let html = "";

  // Previous button
  html += `<button class="page-btn" ${currentPage === 1 ? 'disabled'
: ""} data-page="prev"><</button>`;

  // Page numbers
  for (let i = 1; i <= totalPages; i++) {
    if (i === 1 || i === totalPages || (i >= currentPage - 2 && i <=
currentPage + 2)) {
      html += `<button class="page-btn" ${i === currentPage ?
'active' : ""} data-page="${i}">${i}</button>`;
    } else if (i === currentPage - 3 || i === currentPage + 3) {
      html += '<span class="page-ellipsis">...</span>';
    }
  }
}
```

ASSIGNMENT – 1 & 2

```
// Next button

html += `<button class="page-btn" ${currentPage === totalPages ?
'disabled' : ''} data-page="next">></button>`;

paginationContainer.innerHTML = html;

// Add event listeners to pagination buttons
paginationContainer.querySelectorAll('.page-btn[data-
page]').forEach(btn => {
    btn.addEventListener('click', function() {
        const page = this.dataset.page;
        if (page === 'prev' && currentPage > 1) {
            currentPage--;
        } else if (page === 'next' && currentPage < totalPages) {
            currentPage++;
        } else if (!isNaN(page)) {
            currentPage = parseInt(page);
        }
        renderTable();
    });
});

}

// Filter data based on search and filters
```

ASSIGNMENT – 1 & 2

```
function filterData() {  
    const searchTerm = searchInput ? searchInput.value.toLowerCase() : "";  
    const gradeValue = gradeFilter ? gradeFilter.value : "";  
    const majorValue = majorFilter ? majorFilter.value : "";  
  
    filteredData = currentData.filter(student => {  
        // Search filter  
        const matchesSearch = !searchTerm ||  
            student.firstName.toLowerCase().includes(searchTerm) ||  
            student.lastName.toLowerCase().includes(searchTerm) ||  
            student.email.toLowerCase().includes(searchTerm) ||  
            student.major.toLowerCase().includes(searchTerm);  
  
        // Grade filter  
        const matchesGrade = !gradeValue || student.grade ===  
gradeValue;  
  
        // Major filter  
        const matchesMajor = !majorValue || student.major ===  
majorValue;  
  
        return matchesSearch && matchesGrade && matchesMajor;  
    });  
  
    // Apply sorting
```

ASSIGNMENT – 1 & 2

```
sortData();
```

```
currentPage = 1;
```

```
renderTable();
```

```
}
```

```
// Sort data
```

```
function sortData() {
```

```
  if (!sortSelect) return;
```

```
  const sortBy = sortSelect.value;
```

```
  switch(sortBy) {
```

```
    case 'name-asc':
```

```
      filteredData.sort((a, b) =>  
a.firstName.localeCompare(b.firstName));
```

```
      break;
```

```
    case 'name-desc':
```

```
      filteredData.sort((a, b) =>  
b.firstName.localeCompare(a.firstName));
```

```
      break;
```

```
    case 'age-asc':
```

```
      filteredData.sort((a, b) => a.age - b.age);
```

```
      break;
```

```
    case 'age-desc':
```


ASSIGNMENT – 1 & 2

```
        filteredData.sort((a, b) => b.age - a.age);
        break;
    case 'grade-asc':
        filteredData.sort((a, b) => a.grade.localeCompare(b.grade));
        break;
    case 'grade-desc':
        filteredData.sort((a, b) => b.grade.localeCompare(a.grade));
        break;
    }
}

// Show form for add/edit
function showForm(student = null) {
    if (!studentForm) return;

    if (student) {
        editingId = student.id;
        formTitle.textContent = 'Edit Student';
        document.getElementById('student-id').value = student.id;
        document.getElementById('first-name').value = student.firstName;
        document.getElementById('last-name').value = student.lastName;
        document.getElementById('student-age').value = student.age;
        document.getElementById('student-grade').value = student.grade;
        document.getElementById('student-email').value = student.email;
        document.getElementById('student-major').value = student.major;
```

ASSIGNMENT – 1 & 2

```
} else {  
    editingId = null;  
    formTitle.textContent = 'Add New Student';  
    document.getElementById('student-form').reset();  
}  
  
studentForm.style.display = 'block';  
studentForm.scrollIntoView({ behavior: 'smooth' });  
}  
  
// Hide form  
function hideForm() {  
    if (studentForm) {  
        studentForm.style.display = 'none';  
        editingId = null;  
    }  
}  
  
// Save student (add or update)  
function saveStudent(formData) {  
    const student = {  
        firstName: formData.get('firstName'),  
        lastName: formData.get('lastName'),  
        age: parseInt(formData.get('age')),  
        grade: formData.get('grade'),
```

ASSIGNMENT – 1 & 2

```
email: formData.get('email'),
major: formData.get('major')
};

if (editingId) {
  // Update existing
  const index = currentData.findIndex(s => s.id === editingId);
  if (index !== -1) {
    student.id = editingId;
    currentData[index] = student;
  }
} else {
  // Add new
  const newId = Math.max(...currentData.map(s => s.id), 0) + 1;
  student.id = newId;
  currentData.push(student);
}

filterData();
hideForm();
}

// Delete student
function deleteStudent(id) {
  if (confirm('Are you sure you want to delete this student?')) {
```

ASSIGNMENT – 1 & 2

```
currentData = currentData.filter(s => s.id !== id);
filterData();
}
}

// Export to CSV
function exportToCSV() {
    const headers = ['ID', 'First Name', 'Last Name', 'Age', 'Grade', 'Email',
'Major'];
    const rows = filteredData.map(s => [
        s.id, s.firstName, s.lastName, s.age, s.grade, s.email, s.major
    ]);

    let csv = headers.join(',') + '\n';
    rows.forEach(row => {
        csv += row.map(cell => `"${cell}"`).join(',') + '\n';
    });

    const blob = new Blob([csv], { type: 'text/csv' });
    const url = URL.createObjectURL(blob);
    const a = document.createElement('a');
    a.href = url;
    a.download = 'students.csv';
    a.click();
    URL.revokeObjectURL(url);
}
```

ASSIGNMENT – 1 & 2

```
}
```

```
// Event Listeners
```

```
if (searchInput) {
```

```
    searchInput.addEventListener('input', filterData);
```

```
}
```

```
if (gradeFilter) {
```

```
    gradeFilter.addEventListener('change', filterData);
```

```
}
```

```
if (majorFilter) {
```

```
    majorFilter.addEventListener('change', filterData);
```

```
}
```

```
if (sortSelect) {
```

```
    sortSelect.addEventListener('change', filterData);
```

```
}
```

```
if (addStudentBtn) {
```

```
    addStudentBtn.addEventListener('click', () => showForm());
```

```
}
```

```
if (cancelFormBtn) {
```

```
    cancelFormBtn.addEventListener('click', hideForm);
```

ASSIGNMENT – 1 & 2

```
}
```

```
if (studentForm) {  
  studentForm.addEventListener('submit', function(e) {  
    e.preventDefault();  
    saveStudent(new FormData(this));  
  });  
}
```

```
if (exportBtn) {  
  exportBtn.addEventListener('click', exportToCSV);  
}
```

```
// Event delegation for edit and delete buttons
```

```
if (tableBody) {  
  tableBody.addEventListener('click', function(e) {  
    const target = e.target;  
  
    if (target.classList.contains('edit-btn')) {  
      const id = parseInt(target.dataset.id);  
      const student = currentData.find(s => s.id === id);  
      if (student) showForm(student);  
    }  
  
    if (target.classList.contains('delete-btn')) {
```

ASSIGNMENT – 1 & 2

```
const id = parseInt(target.dataset.id);
deleteStudent(id);
}
});
}
```

```
// Clear filters button
const clearFiltersBtn = document.getElementById('clear-filters');
if (clearFiltersBtn) {
  clearFiltersBtn.addEventListener('click', function() {
    if (searchInput) searchInput.value = "";
    if (gradeFilter) gradeFilter.value = "";
    if (majorFilter) majorFilter.value = "";
    if (sortSelect) sortSelect.value = 'name-asc';
    filterData();
  });
}
```

```
// Initialize
initializeFilters();
filterData();
});
```

```
// HTML structure for this program:
```

```
/*
```

ASSIGNMENT – 1 & 2

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>Student Management Table</title>
```

```
  <style>
```

```
    * { box-sizing: border-box; margin: 0; padding: 0; }
```

```
    body {
```

```
      font-family: 'Segoe UI', Arial, sans-serif;
```

```
      padding: 20px;
```

```
      background: #f5f5f5;
```

```
    }
```

```
    .container { max-width: 1400px; margin: 0 auto; }
```

```
    h1 { margin-bottom: 20px; color: #333; }
```

```
    .controls {
```

```
      display: flex;
```

```
      flex-wrap: wrap;
```

```
      gap: 15px;
```

```
      margin-bottom: 20px;
```

```
      padding: 20px;
```

```
      background: white;
```

```
      border-radius: 8px;
```

```
      box-shadow: 0 2px 4px rgba(0,0,0,0.1);
```

```
    }
```


ASSIGNMENT – 1 & 2

```
.search-box { flex: 2; min-width: 250px; }
```

```
.search-box input {  
  width: 100%;  
  padding: 10px;  
  border: 1px solid #ddd;  
  border-radius: 4px;  
  font-size: 14px;  
}
```

```
.filter-group {  
  display: flex;  
  gap: 10px;  
  flex-wrap: wrap;  
}
```

```
.filter-group select {  
  padding: 10px;  
  border: 1px solid #ddd;  
  border-radius: 4px;  
  font-size: 14px;  
  min-width: 150px;  
}
```

```
.action-buttons {  
  display: flex;
```

ASSIGNMENT – 1 & 2

```
gap: 10px;  
}
```

```
.btn {  
  padding: 10px 20px;  
  border: none;  
  border-radius: 4px;  
  font-size: 14px;  
  cursor: pointer;  
  transition: background 0.3s;  
}
```

```
.btn-primary {  
  background: #2196F3;  
  color: white;  
}
```

```
.btn-primary:hover { background: #1976D2; }
```

```
.btn-success {  
  background: #4CAF50;  
  color: white;  
}
```

```
.btn-success:hover { background: #388E3C; }
```

```
.btn-secondary {
```

ASSIGNMENT – 1 & 2

```
background: #757575;
color: white;
}

.btn-secondary:hover { background: #616161; }

.stats-container {
  display: flex;
  gap: 15px;
  margin-bottom: 20px;
}

.stat-card {
  flex: 1;
  background: white;
  padding: 15px;
  border-radius: 8px;
  text-align: center;
  box-shadow: 0 2px 4px rgba(0,0,0,0.1);
}

.stat-value {
  display: block;
  font-size: 28px;
  font-weight: bold;
  color: #2196F3;
```

ASSIGNMENT – 1 & 2

```
}
```

```
.stat-label {  
    color: #666;  
    font-size: 14px;  
}
```

```
.table-container {  
    background: white;  
    border-radius: 8px;  
    overflow-x: auto;  
    box-shadow: 0 2px 4px rgba(0,0,0,0.1);  
    margin-bottom: 20px;  
}
```

```
table {  
    width: 100%;  
    border-collapse: collapse;  
}
```

```
th {  
    background: #f5f5f5;  
    padding: 12px;  
    text-align: left;  
    font-weight: 600;
```

ASSIGNMENT – 1 & 2

```
border-bottom: 2px solid #ddd;
}

td {
  padding: 12px;
  border-bottom: 1px solid #eee;
}

.grade-badge {
  display: inline-block;
  padding: 4px 8px;
  border-radius: 4px;
  font-weight: bold;
  font-size: 12px;
}

.grade-a { background: #4CAF50; color: white; }
.grade-b { background: #FFC107; color: #333; }
.grade-c { background: #f44336; color: white; }

.action-cell { white-space: nowrap; }

.edit-btn, .delete-btn {
  padding: 5px 10px;
  margin: 0 3px;
```

ASSIGNMENT – 1 & 2

```
border: none;
border-radius: 3px;
cursor: pointer;
font-size: 14px;
}
```

```
.edit-btn { background: #2196F3; color: white; }
.delete-btn { background: #f44336; color: white; }
.edit-btn:hover { background: #1976D2; }
.delete-btn:hover { background: #d32f2f; }
```

```
.pagination {
  display: flex;
  justify-content: center;
  gap: 5px;
  margin-top: 20px;
}
```

```
.page-btn {
  padding: 8px 12px;
  border: 1px solid #ddd;
  background: white;
  cursor: pointer;
  border-radius: 4px;
}
```

ASSIGNMENT – 1 & 2

```
.page-btn:hover { background: #f5f5f5; }
```

```
.page-btn.active {  
    background: #2196F3;  
    color: white;  
    border-color: #2196F3;  
}
```

```
.page-btn[disabled] {  
    opacity: 0.5;  
    cursor: not-allowed;  
}
```

```
.form-container {  
    background: white;  
    padding: 20px;  
    border-radius: 8px;  
    box-shadow: 0 2px 4px rgba(0,0,0,0.1);  
    margin-top: 20px;  
    display: none;  
}
```

```
.form-row {  
    display: grid;  
    grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));  
    gap: 15px;
```

ASSIGNMENT – 1 & 2

```
margin-bottom: 15px;
}

/* Additional CSS for the table */
.form-group label {
    display: block;
    margin-bottom: 5px;
    font-weight: 600;
    color: #333;
}

.form-group input,
.form-group select {
    width: 100%;
    padding: 10px;
    border: 1px solid #ddd;
    border-radius: 4px;
    font-size: 14px;
}

.form-actions {
    display: flex;
    gap: 10px;
    justify-content: flex-end;
    margin-top: 20px;
```


ASSIGNMENT – 1 & 2

```
}
```

```
.page-ellipsis {  
    padding: 8px 4px;  
    color: #999;  
}
```

```
tr:hover {  
    background-color: #f9f9f9 !important;  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="container">
```

```
<h1>  Student Management System</h1>
```

```
<div class="controls">
```

```
<div class="search-box">
```

```
<input type="text" id="search-input" placeholder="  Search by  
name, email, or major...">
```

```
</div>
```

```
<div class="filter-group">
```

```
<select id="grade-filter">
```

```
<option value="">All Grades</option>
```

```
</select>
```

ASSIGNMENT – 1 & 2

```
<select id="major-filter">
  <option value="">All Majors</option>
</select>

<select id="sort-select">
  <option value="name-asc">Name (A-Z)</option>
  <option value="name-desc">Name (Z-A)</option>
  <option value="age-asc">Age (Youngest)</option>
  <option value="age-desc">Age (Oldest)</option>
  <option value="grade-asc">Grade (Low-High)</option>
  <option value="grade-desc">Grade (High-Low)</option>
</select>

</div>

<div class="action-buttons">
  <button class="btn btn-secondary" id="clear-
filters">Clear</button>

  <button class="btn btn-primary" id="add-student-btn">+ Add
Student</button>

  <button class="btn btn-success" id="export-btn">  Export
CSV</button>
</div>

</div>

<div class="stats-container" id="stats-display"> </div>

<div class="table-container">
  <table>
```

ASSIGNMENT – 1 & 2

```
<thead>
  <tr>
    <th>ID</th>
    <th>First Name</th>
    <th>Last Name</th>
    <th>Age</th>
    <th>Grade</th>
    <th>Email</th>
    <th>Major</th>
    <th>Actions</th>
  </tr>
</thead>
<tbody id="table-body"> </tbody>
</table>
</div>

<div class="pagination" id="pagination"> </div>

<div class="form-container" id="student-form">
  <h2 id="form-title">Add New Student</h2>
  <form>
    <input type="hidden" id="student-id">
    <div class="form-row">
      <div class="form-group">
        <label>First Name *</label>
```

ASSIGNMENT – 1 & 2

```
<input type="text" id="first-name" name="firstName"
required>

</div>

<div class="form-group">
  <label>Last Name *</label>
  <input type="text" id="last-name" name="lastName"
required>

</div>
</div>

<div class="form-row">
  <div class="form-group">
    <label>Age *</label>
    <input type="number" id="student-age" name="age"
min="16" max="100" required>
  </div>
  <div class="form-group">
    <label>Grade *</label>
    <select id="student-grade" name="grade" required>
      <option value="">Select Grade</option>
      <option value="A">A</option>
      <option value="A-">A-</option>
      <option value="B+">B+</option>
      <option value="B">B</option>
      <option value="B-">B-</option>
      <option value="C+">C+</option>
      <option value="C">C</option>
    </select>
  </div>
</div>
```

ASSIGNMENT – 1 & 2

```
<option value="F">F</option>
</select>
</div>
</div>
<div class="form-row">
  <div class="form-group">
    <label>Email *</label>
    <input type="email" id="student-email" name="email"
required>
  </div>
  <div class="form-group">
    <label>Major *</label>
    <select id="student-major" name="major" required>
      <option value="">Select Major</option>
      <option value="Computer Science">Computer
Science</option>
      <option value="Mathematics">Mathematics</option>
      <option value="Physics">Physics</option>
      <option value="Engineering">Engineering</option>
      <option value="Biology">Biology</option>
      <option value="Chemistry">Chemistry</option>
    </select>
  </div>
</div>
<div class="form-actions">
```

ASSIGNMENT – 1 & 2

```
<button type="button" class="btn btn-secondary" id="cancel-  
form">Cancel</button>
```

```
<button type="submit" class="btn btn-primary">Save  
Student</button>
```

```
</div>
```

```
</form>
```

```
</div>
```

```
</div>
```

```
<script src="program57.js"></script>
```

```
</body>
```

```
</html>
```

```
*/
```

REGULAR EXPRESSIONS - PATTERN MATCHING AND VALIDATION

// Program 58: Comprehensive Regular Expression examples

```
console.log("--- Creating Regular Expressions ---");
```

```
// Two ways to create RegEx
```

```
const regex1 = /pattern/gi;          // Literal notation
```

```
const regex2 = new RegExp('pattern', 'gi'); // Constructor
```

```
console.log("Literal:", regex1);
```

```
console.log("Constructor:", regex2);
```

ASSIGNMENT – 1 & 2

```
console.log("\n--- Basic Pattern Matching ---");
```

```
// test() - returns boolean
```

```
const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
```

```
console.log("Valid email test:", emailRegex.test("user@example.com")); // true
```

```
console.log("Invalid email test:", emailRegex.test("invalid-email")); // false
```

```
// exec() - returns match details
```

```
const phoneRegex = /(\d{3})-(\d{3})-(\d{4})/;
```

```
const phoneMatch = phoneRegex.exec("Call me at 123-456-7890");
```

```
console.log("Phone match:", phoneMatch);
```

```
console.log("Area code:", phoneMatch[1]);
```

```
console.log("Full match:", phoneMatch[0]);
```

```
console.log("\n--- String Methods with RegEx ---");
```

```
// match() - returns matches
```

```
const text = "The quick brown fox jumps over the lazy dog";
```

```
const vowelMatches = text.match(/[aeiou]/gi);
```

```
console.log("All vowels:", vowelMatches);
```

```
console.log("Vowel count:", vowelMatches.length);
```

```
// matchAll() - returns iterator of all matches
```

```
const dates = "2024-01-15, 2024-02-20, 2024-03-25";
```

ASSIGNMENT – 1 & 2

```
const datePattern = /(\d{4})-(\d{2})-(\d{2})/g;
const dateMatches = [...dates.matchAll(datePattern)];
console.log("\nDate matches:");
dateMatches.forEach(match => {
    console.log(` Year: ${match[1]}, Month: ${match[2]}, Day:
    ${match[3]}`);
});

// replace() - replace matches
const censored = "This is a bad word in a sentence".replace(/bad/, "****");
console.log("\nCensored:", censored);

// replace with callback
const prices = "Items: $10.99, $25.50, $100.00";
const withTax = prices.replace(/\$(\d+\.\d{2})/g, (match, price) => {
    const withTax = parseFloat(price) * 1.1;
    return `$$${withTax.toFixed(2)}`;
});
console.log("With 10% tax:", withTax);

// split() - split by regex
const csv = "apple,banana;orange|grape";
const fruits = csv.split(/[,;|]/);
console.log("\nSplit result:", fruits);
```


ASSIGNMENT – 1 & 2

```
// search() - returns index of first match
```

```
const index = "Hello World".search(/world/i);
```

```
console.log("'World' found at index:", index);
```

```
console.log("\n--- RegEx Flags ---");
```

```
const flagsDemo = "Apple apple APPLE";
```

```
console.log("No flags:", flagsDemo.match(/apple/));    // ['apple']
```

```
console.log("i flag:", flagsDemo.match(/apple/i));    // ['Apple']
```

```
console.log("g flag:", flagsDemo.match(/apple/g));    // ['apple']
```

```
console.log("gi flag:", flagsDemo.match(/apple/gi));  // ['Apple', 'apple',  
'APPLE']
```

```
console.log("\n--- Character Classes ---");
```

```
// \d - digits
```

```
console.log("Digits in 'abc123def456!':", 'abc123def456'.match(/\d+/g));
```

```
// \w - word characters (letters, numbers, underscore)
```

```
console.log("Words in 'hello_world 123!':", 'hello_world  
123!'.match(/\w+/g));
```

```
// \s - whitespace
```

```
console.log("Whitespace count in 'a b c':", 'a b\tc\n'.match(/\s/g).length);
```

ASSIGNMENT – 1 & 2

// \b - word boundary

```
console.log("Word 'cat':", 'The cat scattered'.match(/\bcat\b/g));
```

// Negated classes

```
console.log("Non-digits:", 'abc123def'.match(/\D+/g));
```

```
console.log("\n--- Quantifiers ---");
```

// * (0 or more), + (1 or more), ? (0 or 1)

```
console.log("a*:", 'aaa'.match(/a*/g));
```

```
console.log("a+:", 'aaa'.match(/a+/g));
```

```
console.log("a?:", 'aaa'.match(/a?/g));
```

// {n} exactly n, {n,} at least n, {n,m} between n and m

```
console.log("3 digits:", '12345'.match(/\d{3}/g));
```

```
console.log("2-4 digits:", '1 12 123 1234 12345'.match(/\d{2,4}/g));
```

```
console.log("\n--- Groups and Capturing ---");
```

// Capturing groups (...)

```
const nameRegex = /(\w+)\s+(\w+)/;
```

```
const nameMatch = "John Doe".match(nameRegex);
```

```
console.log("Full name:", nameMatch[0]);
```

```
console.log("First:", nameMatch[1]);
```

```
console.log("Last:", nameMatch[2]);
```

ASSIGNMENT – 1 & 2

```
// Named capturing groups (?<name>...)
const namedRegex = /(?!<first>\w+)\s+(?!<last>\w+)/;
const namedMatch = "Jane Smith".match(namedRegex);
console.log("\nNamed groups:");
console.log("First:", namedMatch.groups.first);
console.log("Last:", namedMatch.groups.last);

// Non-capturing groups (?:...)
const nonCapture = /(?:https?:\V\V)?(?:www\.)?(\w+\.\w+)/;
const urlMatch = "https://www.example.com".match(nonCapture);
console.log("\nDomain only:", urlMatch[1]);

// Backreferences \1, \2
const repeated = "the the word word repeated".match(/(\w+)\s+\1/g);
console.log("Repeated words:", repeated);

console.log("\n--- Lookahead and Lookbehind ---");

// Positive lookahead (?=...)
console.log("Digits followed by $:");
console.log("$100 $50 $25".match(/\d+(?=\$/g)); // Empty (no matches)

// Negative lookahead (?!...)
console.log("Digits NOT followed by $:");
```

ASSIGNMENT – 1 & 2

```
console.log("100$ 50 25$".match(/\d+(?!\$)/g));
```

```
// Positive lookbehind (?<=...)
```

```
console.log("Digits after $:");
```

```
console.log("$100 $50 25$".match(/(?<=\$)\d+/g));
```

```
console.log("\n--- Practical Validation Examples ---");
```

```
const validators = {
```

```
  // Email validation
```

```
  email: (value) => /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/i.test(value),
```

```
  // Phone (US format)
```

```
  phone: (value) => /^(?(\d{3})?[-.\s]?(\d{3})[-.\s]?(\d{4})$/i.test(value),
```

```
  // URL
```

```
  url: (value) => /^(https?:\/\/)?([\da-z.-]+)\.([a-z.]{2,6})([/\w .-]*)*\/?$/i.test(value),
```

```
  // Password (8+ chars, 1 upper, 1 lower, 1 number)
```

```
  password: (value) => /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)[a-zA-Z\d]{8,}$/i.test(value),
```

```
  // ZIP code
```

```
  zipCode: (value) => /^\d{5}(-\d{4})?$/i.test(value),
```

ASSIGNMENT – 1 & 2

```
// Date (YYYY-MM-DD)
date: (value) => /^\\d{4}-(0[1-9]|1[0-2])-(0[1-9]|1[2]\\d|3[01])$/.test(value),

// Hex color
hexColor: (value) => /^#?([a-fA-F0-9]{6}|[a-fA-F0-9]{3})$/.test(value),

// Username (alphanumeric, 3-20 chars)
username: (value) => /^[a-zA-Z0-9_]{3,20}$/.test(value)
};

console.log("\\nValidation Tests:");
const tests = [
  { type: 'email', value: 'user@example.com', expected: true },
  { type: 'email', value: 'invalid-email', expected: false },
  { type: 'phone', value: '(123) 456-7890', expected: true },
  { type: 'phone', value: '123.456.7890', expected: true },
  { type: 'password', value: 'Password123', expected: true },
  { type: 'password', value: 'weak', expected: false },
  { type: 'url', value: 'https://example.com', expected: true },
  { type: 'hexColor', value: '#FF5733', expected: true },
  { type: 'zipCode', value: '12345-6789', expected: true }
];
```

ASSIGNMENT – 1 & 2

```
tests.forEach(test => {  
  const result = validators[test.type](test.value);  
  const status = result === test.expected ? '✓' : '✗';  
  console.log(` ${status} ${test.type}: "${test.value}" = ${result}`);  
});
```

```
console.log("\n--- Extracting Data from Text ---");
```

```
const sampleText = `
```

```
Contact Information:
```

```
John Doe - john.doe@email.com - (555) 123-4567
```

```
Jane Smith - jane.smith@company.co.uk - 555.987.6543
```

```
Bob Johnson - bjohnson@work.org - (555) 555-5555
```

```
`;
```

```
// Extract all emails
```

```
const emails = sampleText.match(/[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/g);
```

```
console.log("Extracted emails:", emails);
```

```
// Extract all phone numbers
```

```
const phones = sampleText.match(/\(?\d{3}\)?[-.\s]? \d{3}[-.\s]? \d{4}/g);
```

```
console.log("Extracted phones:", phones);
```

```
// Extract names and create objects
```

ASSIGNMENT – 1 & 2

```
const contactPattern = /(\w+\s+\w+)\s*-\s*(\S+@\S+)\s*-\s*([\d\s()\-]+)/g;
const contacts = [...sampleText.matchAll(contactPattern)].map(match => ({
  name: match[1],
  email: match[2],
  phone: match[3].trim()
}));
console.log("Parsed contacts:", contacts);
```

FORM VALIDATION WITH REGEX AND DOM

// Program 59: Complete form validation system

```
document.addEventListener('DOMContentLoaded', function() {

  // Validation rules configuration
  const validationRules = {
    username: {
      required: true,
      minLength: 3,
      maxLength: 20,
      pattern: /^[a-zA-Z0-9_]+$/,
      message: 'Username must be 3-20 alphanumeric characters or underscore'
    },
    email: {
      required: true,
      pattern: /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/,
```

ASSIGNMENT – 1 & 2

```
message: 'Please enter a valid email address'
},
password: {
  required: true,
  minLength: 8,
  pattern: /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)[a-zA-Z\d@$_!%*?&]{8,}$/,
  message: 'Password must be 8+ chars with uppercase, lowercase,
and number'
},
confirmPassword: {
  required: true,
  matchWith: 'password',
  message: 'Passwords do not match'
},
phone: {
  required: false,
  pattern: /^\(?\d{3}\)?[-.\s]?\d{3}[-.\s]?\d{4}$/,
  message: 'Enter valid phone: (123) 456-7890 or 123-456-7890'
},
age: {
  required: true,
  min: 18,
  max: 120,
  message: 'Age must be between 18 and 120'
```


ASSIGNMENT – 1 & 2

```
},  
website: {  
  required: false,  
  pattern: /^(https?:\V)?([\da-z.-]+)\.([a-z.]{2,6})([\w .-]*)*V?$/,  
  message: 'Enter a valid URL (e.g., https://example.com)'  
},  
zipCode: {  
  required: true,  
  pattern: /^\d{5}(-\d{4})?$/,  
  message: 'Enter valid ZIP code: 12345 or 12345-6789'  
},  
creditCard: {  
  required: false,  
  pattern: /^\d{4}[\s-]?){3}\d{4}$/,  
  message: 'Enter valid 16-digit card number'  
}  
};
```

```
// Get form and fields
```

```
const form = document.getElementById('validation-form');  
const fields = document.querySelectorAll('[data-validate]');  
const submitBtn = document.getElementById('submit-btn');  
const resetBtn = document.getElementById('reset-btn');  
const successMessage = document.getElementById('success-message');
```

ASSIGNMENT – 1 & 2

```
// Validation state
const fieldState = new Map();

// Initialize fields
fields.forEach(field => {
  fieldState.set(field.id, {
    touched: false,
    valid: false,
    errors: []
  });
});

// Validate a single field
function validateField(field) {
  const fieldId = field.id;
  const rules = validationRules[fieldId];
  const value = field.value.trim();
  const state = fieldState.get(fieldId);
  const errors = [];

  if (!rules) return true;

  // Required check
  if (rules.required && !value) {
    errors.push(` ${fieldId} is required`);
  }
}
```

ASSIGNMENT – 1 & 2

```
}
```

```
if (value) {
```

```
    // Min length
```

```
    if (rules.minLength && value.length < rules.minLength) {
```

```
        errors.push(`Minimum ${rules.minLength} characters required`);
```

```
    }
```

```
    // Max length
```

```
    if (rules.maxLength && value.length > rules.maxLength) {
```

```
        errors.push(`Maximum ${rules.maxLength} characters allowed`);
```

```
    }
```

```
    // Pattern check
```

```
    if (rules.pattern && !rules.pattern.test(value)) {
```

```
        errors.push(rules.message);
```

```
    }
```

```
    // Min value
```

```
    if (rules.min !== undefined && parseInt(value) < rules.min) {
```

```
        errors.push(rules.message);
```

```
    }
```

```
    // Max value
```

```
    if (rules.max !== undefined && parseInt(value) > rules.max) {
```

ASSIGNMENT – 1 & 2

```
errors.push(rules.message);
}

// Match with another field
if (rules.matchWith) {
  const matchField = document.getElementById(rules.matchWith);
  if (matchField && value !== matchField.value) {
    errors.push(rules.message);
  }
}

// Custom validation for credit card (Luhn algorithm)
if (fieldId === 'creditCard' && value) {
  if (!luhnCheck(value.replace(/\D/g, ""))) {
    errors.push('Invalid credit card number');
  }
}

}

// Update state
state.errors = errors;
state.valid = errors.length === 0;

// Update UI
updateFieldUI(field, errors);
```

ASSIGNMENT – 1 & 2

```
    return state.valid;
}

// Luhn algorithm for credit card validation
function luhnCheck(cardNumber) {
    if (!/^\\d+$/ .test(cardNumber)) return false;

    let sum = 0;
    let alternate = false;

    for (let i = cardNumber.length - 1; i >= 0; i--) {
        let n = parseInt(cardNumber.charAt(i));
        if (alternate) {
            n *= 2;
            if (n > 9) n -= 9;
        }
        sum += n;
        alternate = !alternate;
    }

    return sum % 10 === 0;
}

// Update field UI with validation state
```

ASSIGNMENT – 1 & 2

```
function updateFieldUI(field, errors) {  
  const formGroup = field.closest('.form-group');  
  const errorContainer = formGroup.querySelector('.field-error');  
  const successIcon = formGroup.querySelector('.success-icon');  
  const errorIcon = formGroup.querySelector('.error-icon');  
  
  // Remove existing classes  
  formGroup.classList.remove('valid', 'invalid');  
  
  if (errors.length === 0 && fieldState.get(field.id).touched) {  
    formGroup.classList.add('valid');  
    if (successIcon) successIcon.style.display = 'inline';  
    if (errorIcon) errorIcon.style.display = 'none';  
    if (errorContainer) errorContainer.textContent = "";  
  } else if (errors.length > 0) {  
    formGroup.classList.add('invalid');  
    if (successIcon) successIcon.style.display = 'none';  
    if (errorIcon) errorIcon.style.display = 'inline';  
    if (errorContainer) errorContainer.textContent = errors[0];  
  } else {  
    if (successIcon) successIcon.style.display = 'none';  
    if (errorIcon) errorIcon.style.display = 'none';  
    if (errorContainer) errorContainer.textContent = "";  
  }  
}
```

ASSIGNMENT – 1 & 2

```
// Validate entire form
function validateForm() {
  let isValid = true;

  fields.forEach(field => {
    const state = fieldState.get(field.id);
    state.touched = true;

    if (!validateField(field)) {
      isValid = false;
    }
  });

  // Update submit button
  if (submitBtn) {
    submitBtn.disabled = !isValid;
  }

  return isValid;
}

// Real-time validation on input
fields.forEach(field => {
  field.addEventListener('input', function() {
```

ASSIGNMENT – 1 & 2

```
const state = fieldState.get(this.id);  
if (state.touched) {  
    validateField(this);  
}  
validateForm();  
});
```

```
field.addEventListener('blur', function() {  
    const state = fieldState.get(this.id);  
    state.touched = true;  
    validateField(this);  
    validateForm();  
});
```

```
field.addEventListener('focus', function() {  
    const formGroup = this.closest('.form-group');  
    formGroup.classList.add('focused');  
});  
});
```

// Password strength indicator

```
const passwordField = document.getElementById('password');  
const strengthBar = document.getElementById('password-strength');  
const strengthText = document.getElementById('strength-text');
```


ASSIGNMENT – 1 & 2

```
if (passwordField) {  
  passwordField.addEventListener('input', function() {  
    const password = this.value;  
    const strength = calculatePasswordStrength(password);  
  
    if (strengthBar) {  
      strengthBar.style.width = `${strength.score * 25}%`;   
      strengthBar.style.backgroundColor = strength.color;  
    }  
  
    if (strengthText) {  
      strengthText.textContent = strength.label;  
      strengthText.style.color = strength.color;  
    }  
  });  
}  
  
function calculatePasswordStrength(password) {  
  if (!password) return { score: 0, label: 'No password', color: '#ddd' };  
  
  let score = 0;  
  
  if (password.length >= 8) score++;  
  if (password.length >= 12) score++;  
  if (/[a-z]/.test(password)) score++;
```

ASSIGNMENT – 1 & 2

```
if (/[A-Z]/.test(password)) score++;
```

```
if (/^d/.test(password)) score++;
```

```
if (/^[^a-zA-Z0-9]/.test(password)) score++;
```

```
const normalized = Math.min(Math.floor(score / 1.5), 4);
```

```
const levels = [
```

```
  { label: 'Very Weak', color: '#f44336' },
```

```
  { label: 'Weak', color: '#ff9800' },
```

```
  { label: 'Fair', color: '#ffc107' },
```

```
  { label: 'Good', color: '#2196F3' },
```

```
  { label: 'Strong', color: '#4CAF50' }
```

```
];
```

```
return { score: normalized, ...levels[normalized] };
```

```
}
```

```
// Form submission
```

```
if (form) {
```

```
  form.addEventListener('submit', function(e) {
```

```
    e.preventDefault();
```

```
    if (validateForm()) {
```

```
      // Collect form data
```

```
      const formData = new FormData(form);
```

ASSIGNMENT – 1 & 2

```
const data = {};  
formData.forEach((value, key) => {  
    data[key] = value;  
});  
  
console.log('Form submitted successfully:', data);  
  
// Show success message  
if (successMessage) {  
    successMessage.style.display = 'block';  
    successMessage.textContent = '✓ Form submitted successfully!';  
}  
  
// Optional: Disable form after success  
// form.querySelectorAll('input, select, button').forEach(el =>  
el.disabled = true);  
} else {  
    if (successMessage) {  
        successMessage.style.display = 'block';  
        successMessage.style.backgroundColor = '#ffebee';  
        successMessage.style.color = '#c62828';  
        successMessage.textContent = 'X Please fix validation errors!';  
    }  
}  
});
```

ASSIGNMENT – 1 & 2

```
}
```

```
// Reset form
```

```
if (resetBtn) {
```

```
  resetBtn.addEventListener('click', function() {
```

```
    form.reset();
```

```
    fields.forEach(field => {
```

```
      const state = fieldState.get(field.id);
```

```
      state.touched = false;
```

```
      state.valid = false;
```

```
      state.errors = [];
```

```
      const formGroup = field.closest('.form-group');
```

```
      formGroup.classList.remove('valid', 'invalid', 'focused');
```

```
      const successIcon = formGroup.querySelector('.success-icon');
```

```
      const errorIcon = formGroup.querySelector('.error-icon');
```

```
      if (successIcon) successIcon.style.display = 'none';
```

```
      if (errorIcon) errorIcon.style.display = 'none';
```

```
    });
```

```
  if (submitBtn) submitBtn.disabled = true;
```

```
  if (successMessage) successMessage.style.display = 'none';
```

```
  if (strengthBar) strengthBar.style.width = '0%';
```

ASSIGNMENT – 1 & 2

```
        if (strengthText) strengthText.textContent = "";
    });
}

// Initialize submit button state
if (submitBtn) {
    submitBtn.disabled = true;
}
});

// HTML structure for this program:
/*
<!DOCTYPE html>
<html>
<head>
    <title>Form Validation with RegEx</title>
    <style>
        * { box-sizing: border-box; margin: 0; padding: 0; }
        body {
            font-family: 'Segoe UI', Arial, sans-serif;
            background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
            min-height: 100vh;
            display: flex;
            justify-content: center;
            align-items: center;
```

ASSIGNMENT – 1 & 2

```
padding: 20px;  
}
```

```
.form-container {  
  background: white;  
  border-radius: 16px;  
  box-shadow: 0 20px 60px rgba(0,0,0,0.3);  
  padding: 40px;  
  max-width: 600px;  
  width: 100%;  
}
```

```
h2 {  
  margin-bottom: 30px;  
  color: #333;  
  text-align: center;  
}
```

```
.form-group {  
  margin-bottom: 24px;  
  position: relative;  
}
```

```
.form-group label {  
  display: block;
```

ASSIGNMENT – 1 & 2

```
margin-bottom: 8px;
font-weight: 600;
color: #555;
font-size: 14px;
}

.input-wrapper {
  position: relative;
}

.form-group input {
  width: 100%;
  padding: 12px 40px 12px 16px;
  border: 2px solid #e0e0e0;
  border-radius: 8px;
  font-size: 16px;
  transition: all 0.3s;
}

.form-group input:focus {
  outline: none;
  border-color: #667eea;
  box-shadow: 0 0 0 3px rgba(102, 126, 234, 0.1);
}
```

ASSIGNMENT – 1 & 2

```
.form-group.valid input {  
    border-color: #4CAF50;  
}
```

```
.form-group.invalid input {  
    border-color: #f44336;  
}
```

```
.validation-icon {  
    position: absolute;  
    right: 12px;  
    top: 50%;  
    transform: translateY(-50%);  
    font-size: 18px;  
    display: none;  
}
```

```
.form-group.valid .success-icon {  
    display: inline;  
    color: #4CAF50;  
}
```

```
.form-group.invalid .error-icon {  
    display: inline;  
    color: #f44336;
```


ASSIGNMENT – 1 & 2

```
}
```

```
.field-error {  
    color: #f44336;  
    font-size: 12px;  
    margin-top: 5px;  
    min-height: 18px;  
}
```

```
.password-strength-container {  
    margin-top: 8px;  
}
```

```
.strength-bar-bg {  
    height: 4px;  
    background: #e0e0e0;  
    border-radius: 2px;  
    overflow: hidden;  
}
```

```
.strength-bar {  
    height: 100%;  
    width: 0%;  
    transition: width 0.3s, background 0.3s;  
}
```

ASSIGNMENT – 1 & 2

```
.strength-text {  
    font-size: 12px;  
    margin-top: 4px;  
    text-align: right;  
}
```

```
.form-row {  
    display: grid;  
    grid-template-columns: 1fr 1fr;  
    gap: 16px;  
}
```

```
.form-actions {  
    display: flex;  
    gap: 16px;  
    margin-top: 32px;  
}
```

```
.btn {  
    flex: 1;  
    padding: 14px;  
    border: none;  
    border-radius: 8px;  
    font-size: 16px;
```

ASSIGNMENT – 1 & 2

```
font-weight: 600;
cursor: pointer;
transition: all 0.3s;
}

.btn-primary {
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  color: white;
}

.btn-primary:hover:not(:disabled) {
  transform: translateY(-2px);
  box-shadow: 0 5px 20px rgba(102, 126, 234, 0.4);
}

.btn-primary:disabled {
  opacity: 0.5;
  cursor: not-allowed;
}

.btn-secondary {
  background: #f5f5f5;
  color: #666;
}
```

ASSIGNMENT – 1 & 2

```
.btn-secondary:hover {  
    background: #e0e0e0;  
}
```

```
.success-message {  
    margin-top: 20px;  
    padding: 15px;  
    border-radius: 8px;  
    text-align: center;  
    display: none;  
}
```

```
.success-message.show {  
    display: block;  
    background: #e8f5e9;  
    color: #2e7d32;  
}
```

```
.required::after {  
    content: " *";  
    color: #f44336;  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

ASSIGNMENT – 1 & 2

```
<div class="form-container">
```

```
<h2><img alt="pencil icon" data-bbox="238 128 265 148"/> Registration Form</h2>
```

```
<form id="validation-form">
```

```
<div class="form-group">
```

```
<label class="required">Username</label>
```

```
<div class="input-wrapper">
```

```
<input type="text" id="username" name="username" data-validate placeholder="johndoe123">
```

```
<span class="validation-icon success-icon">✓</span>
```

```
<span class="validation-icon error-icon">✗</span>
```

```
</div>
```

```
<div class="field-error"></div>
```

```
</div>
```

```
<div class="form-group">
```

```
<label class="required">Email</label>
```

```
<div class="input-wrapper">
```

```
<input type="email" id="email" name="email" data-validate placeholder="john@example.com">
```

```
<span class="validation-icon success-icon">✓</span>
```

```
<span class="validation-icon error-icon">✗</span>
```

```
</div>
```

```
<div class="field-error"></div>
```

```
</div>
```

ASSIGNMENT – 1 & 2

```
<div class="form-group">
  <label class="required">Password</label>
  <div class="input-wrapper">
    <input type="password" id="password" name="password"
data-validate placeholder="••••••••">
    <span class="validation-icon success-icon">✓</span>
    <span class="validation-icon error-icon">✗</span>
  </div>
  <div class="password-strength-container">
    <div class="strength-bar-bg">
      <div class="strength-bar" id="password-strength"></div>
    </div>
    <div class="strength-text" id="strength-text"></div>
  </div>
  <div class="field-error"></div>
</div>
```

```
<div class="form-group">
  <label class="required">Confirm Password</label>
  <div class="input-wrapper">
    <input type="password" id="confirmPassword"
name="confirmPassword" data-validate placeholder="••••••••">
    <span class="validation-icon success-icon">✓</span>
    <span class="validation-icon error-icon">✗</span>
  </div>
</div>
```

ASSIGNMENT – 1 & 2

</div>

<div class="field-error"></div>

</div>

<div class="form-row">

<div class="form-group">

<label>Phone (Optional)</label>

<div class="input-wrapper">

<input type="tel" id="phone" name="phone" data-validate
placeholder="(123) 456-7890">

✓

X

</div>

<div class="field-error"></div>

</div>

<div class="form-group">

<label class="required">Age</label>

<div class="input-wrapper">

<input type="number" id="age" name="age" data-validate
placeholder="18+">

✓

X

</div>

<div class="field-error"></div>

ASSIGNMENT – 1 & 2

</div>

</div>

<div class="form-group">

<label>Website (Optional)</label>

<div class="input-wrapper">

<input type="url" id="website" name="website" data-validate
placeholder="https://example.com">

✓

X

</div>

<div class="field-error"></div>

</div>

<div class="form-row">

<div class="form-group">

<label class="required">ZIP Code</label>

<div class="input-wrapper">

<input type="text" id="zipCode" name="zipCode" data-
validate placeholder="12345 or 12345-6789">

✓

X

</div>

<div class="field-error"></div>

</div>

ASSIGNMENT – 1 & 2

```
<div class="form-group">
  <label>Credit Card (Optional)</label>
  <div class="input-wrapper">
    <input type="text" id="creditCard" name="creditCard"
data-validate placeholder="1234 5678 9012 3456">
    <span class="validation-icon success-icon">✓</span>
    <span class="validation-icon error-icon">X</span>
  </div>
  <div class="field-error"></div>
</div>
</div>

<div class="form-actions">
  <button type="submit" class="btn btn-primary" id="submit-
btn">Register</button>
  <button type="button" class="btn btn-secondary" id="reset-
btn">Reset</button>
</div>

<div class="success-message" id="success-message"></div>
</form>
</div>

<script src="program59.js"></script>
```

ASSIGNMENT – 1 & 2

</body>

</html>

*/

[Navigator Object and Browser Detection](#)

[DOM Tree Traversal - Comprehensive Example](#)

[Advanced DOM 2 Event Handling](#)

[Dynamic Positioning - Absolute and Relative](#)

[Moving Elements with JavaScript Animation](#)

[Element Visibility Control and Toggling](#)

[Z-Index and Stacking Elements](#)

[Mouse Tracking and Cursor Effects](#)

[Smooth Element Movement \(Animation Loop\)](#)

[Drag and Drop Implementation](#)

[XML Parsing and Display with JavaScript](#)

ASSIGNMENT – 1 & 2

[XML with CSS Styling](#)

[XSLT Transformation Example](#)